

Designing Secure Boot for Space Missions

Integrating Security into Spacecraft Bootloaders

Matyas Kollert

Delft University of Technology

Designing Secure Boot for Space Missions

Integrating Security into Spacecraft
Bootloaders

by

Matyas Kollert

to obtain the degree of Master of Science
at the Delft University of Technology,
to be defended publicly on Monday June 22, 2026 at 09:00 AM.

Student number:	5477549
Project duration:	November 8, 2025 – June 22, 2026
Thesis committee:	G. Smaragdakis, TU Delft, thesis advisor A. Voulimeneas, TU Delft, daily supervisor S. Chakraborty, TU Delft A. Atlasis, ESA supervisor Y. Roiron, ESA supervisor

Cover:	Render of Daybreak at Gale Crater by NASA/JPL-Caltech under CC BY 2.0
Style:	TU Delft Report Style, with modifications by Daan Zwaneveld

An electronic version of this thesis is available at <http://repository.tudelft.nl/>.

Abstract

Modern spacecraft increasingly rely on commercial off-the-shelf (COTS) hardware, reducing development cost but expanding the attack surface. The bootloader is the most exposed component: it executes before any other software and, if compromised, enables persistent unauthorised code execution for the entire mission lifetime. Despite this risk, Secure Boot has not seen broad adoption in space, primarily due to stringent reliability requirements, limited resources, and the high cost of qualifying new technology for space.

This thesis investigates how Secure Boot can be incorporated into space mission bootloaders without compromising these constraints. A lifecycle-based threat model, grounded in the ENISA Space Threat Landscape taxonomy, identifies two primary attacker scenarios: a physical adversary with pre-launch hardware access, and a remote adversary exploiting firmware update channels after deployment. These scenarios motivate a formal Secure Boot specification, designated SAVOIR-GS-002S, written in the language and requirement structure of the ESA SAVOIR framework so that it can be adopted by standards bodies without a translation step. The specification introduces seven additive security requirements covering a hardware Root of Trust, digital signature verification, non-bypassability, protection against both classical and quantum computational attacks, rollback protection via a monotonic counter, and structured failure reporting.

A hardware trade-off analysis shows that only two primitives are strictly necessary to realise the specification: sector-level hardware-enforced write protection for the Boot Software and public key storage, and protected non-volatile storage for the monotonic rollback counter. Both are present on commodity ARM Cortex-M microcontrollers without requiring any specialised hardware. A proof-of-concept implementation in C on the STM32F439ZI confirms that all seven requirements are achievable on representative COTS hardware. The implementation includes A/B image partitioning, monotonic rollback protection in option-byte-protected flash, PUS-compatible boot reporting, and an algorithm-agnostic cryptographic verification interface, with an automated end-to-end test suite verifying all nominal and failure paths over UART and SWD.

Performance benchmarks for ECDSA-P256, RSA-3072/4096, ML-DSA-44/65, and LMS demonstrate that the security overhead is modest. A hybrid ECDSA-P256 plus LMS deployment on a 128 kB image completes in approximately 363 ms with hardware SHA-2 acceleration, well within a one-second boot budget, and LMS requires only 3 kB of working memory and a 60-byte public key. The results show that incorporating Secure Boot into space-grade bootloaders is not a fundamental hardware or performance barrier, but primarily a gap in standards and tooling. The SAVOIR-GS-002S specification and accompanying implementation provide a concrete, standards-compatible starting point for closing that gap.

Contents

Abstract	i
Nomenclature	v
1 Introduction	1
1.1 Context - rise of COTS in space and the cybersecurity gap	1
1.2 Problem statement - why bootloaders are the critical attack surface	2
1.3 Research questions	2
1.4 Scope and contributions	2
1.5 Report structure	3
2 Background	4
2.1 Bootloaders	4
2.1.1 Bootloader Types	4
2.1.2 A/B partitioning and OTA updates	4
2.2 Secure boot fundamentals	5
2.2.1 Root of trust	5
2.2.2 Data verification	5
2.2.3 Secure firmware updates	6
2.2.4 Rollback protection	6
2.3 Post-quantum cryptography	7
2.3.1 Motivation and timeline	7
2.4 The space environment and its constraints	8
2.4.1 Radiation tolerance	8
2.4.2 Resource constraints - compute, power, memory	8
2.4.3 SAVOIR - Generic OBC specification	9
2.4.4 SAVOIR - Flight Computer Initialisation Sequence	9
2.4.5 ECSS Standards	10
2.4.6 Cryptographic recommendations for space	11
3 Threat Model	13
3.1 Threat modelling methodology (ENISA lifecycle approach)	13
3.2 Attacker scenarios	13
3.2.1 Physical access prior to launch	13
3.2.2 Remote access after launch	14
3.3 Scope of protection	14
4 Design	16
4.1 Design approach - secure by design, SAVOIR-compatible	16
4.1.1 Existing material	16
4.2 Secure boot specification (SAVOIR-GS-002S extension)	16
4.2.1 Establishing a hardware root of trust	17
4.2.2 Authenticated execution	17
4.2.3 Post-quantum cryptographic support	17
4.2.4 Rollback protection	18
4.2.5 Failure visibility	18
4.3 Key design trade-offs and discarded options	18
4.4 How does this address the threat model?	19
5 PoC Scope and Hardware Selection	20
5.1 From specification to proof of concept	20

5.2	Hardware requirements for space-grade secure boot	21
5.2.1	Immutable storage for RoT and public key	22
5.2.2	Flash and RAM capacity	22
5.2.3	Protected non-volatile storage for security-critical state	22
5.2.4	Communication channel with ground and the ASW	22
5.2.5	Cryptographic acceleration - value and trade-offs	22
5.3	Space-grade hardware overview	22
5.3.1	LEON/SPARC - ESA heritage processors	23
5.3.2	ARM Cortex-M in space - adoption and rationale	23
5.4	Platform selection for the PoC	23
5.4.1	Candidate board comparison	23
5.4.2	Selection rationale - NUCLEO-F439ZI	24
6	Implementation	26
6.1	Secure by design	26
6.2	Memory layout	26
6.2.1	Flash partition structure	26
6.2.2	ASW image format	27
6.2.3	Option byte configuration	28
6.3	Communication	28
6.3.1	Board Support Package	28
6.4	Boot Process	29
6.4.1	State management	29
6.4.2	Nominal mode flow	29
6.4.3	Standby mode - update and report	30
6.4.4	Integrity and signature verification sequence	31
6.5	Rollback protection implementation	32
6.5.1	Option A - version-based window, no counter	32
6.5.2	Option B - two partitions with monotonic counter	33
6.5.3	Option C - three partitions, no counter	33
6.5.4	Selected approach and rationale	33
6.6	Cryptographic interface	34
6.6.1	Algorithm-agnostic verification API	34
6.6.2	Classical algorithms - ECDSA	35
6.6.3	Post-quantum algorithms - ML-DSA	35
6.6.4	Hybrid algorithms - ECDSA + ML-DSA	35
7	Evaluation	36
7.1	Evaluation methodology	36
7.2	Security analysis	36
7.2.1	Coverage of attacker scenarios from threat model	36
7.2.2	Requirement verification (per SEC-* requirements)	37
7.3	Performance benchmarks	39
7.3.1	Benchmark setup - hardware, toolchain, methodology	39
7.3.2	Signature verification latency	40
7.3.3	Classical vs PQ comparison and boot time impact	41
7.3.4	Code size and memory footprint	41
7.4	Limitations and threats to validity	42
7.5	Evaluation summary	43
8	Related Work	44
8.1	Secure boot in terrestrial systems	44
8.1.1	Android Verified Boot	44
8.1.2	UEFI Secure Boot	44
8.1.3	wolfBoot	45
8.2	Security in space systems	45
8.2.1	GRBOOT	45
8.2.2	N7 Space Boot Software	45

9 Conclusion	47
9.1 Summary of contributions	47
9.2 Answers to research questions	48
9.3 Future work	49
9.3.1 Modifying other standards to fully support Secure Boot	49
9.3.2 Trust anchor management and key rotation (RFC 6024, RFC 5280)	49
9.3.3 Partial image updates	50
9.3.4 Secure state management	50
9.3.5 Hardware security module and TrustZone integration	50
9.3.6 Validation on LEON/space-grade hardware	51
9.3.7 Formal verification of boot chain	51
9.3.8 Runtime detection beyond boot-time verification	52
9.4 Reflection	52
9.4.1 Research in the broader computer science field	52
9.4.2 Implications, applicability, and real-world impact	52
9.5 Use of AI	53
References	54
A SAVOIR-GS-002S Secure Boot Generic Specification	58
B wolfSSL Benchmark Configuration	61

Nomenclature

Abbreviations

Abbreviation	Definition
AMP	Asymmetric Multi-Processing
ANSSI	Agence nationale de la sécurité des systèmes d'information
API	Application Programming Interface
ASW	Application Software
BSI	Bundesamt für Sicherheit in der Informationstechnik
BSP	Board Support Package
BSW	Boot Software
C2	Command and Control
CCSDS	Consultative Committee for Space Data Systems
CCMRAM	Core-Coupled Memory RAM
CMAC	Cipher-based Message Authentication Code
CoT	Chain of Trust
COTS	Commercial Off-The-Shelf
CRC	Cyclic Redundancy Check
CRQC	Cryptographically Relevant Quantum Computer
DMA	Direct Memory Access
DSS	Digital Signature Standard
ECSS	European Cooperation for Space Standardization
ECDSA	Elliptic Curve Digital Signature Algorithm
EdDSA	Edwards-curve Digital Signature Algorithm
EEPROM	Electrically Erasable Programmable Read-Only Memory
ELF	Executable and Linkable Format
ENISA	European Union Agency for Cybersecurity
ESA	European Space Agency
FDIR	Fault Detection, Isolation and Recovery
FIPS	Federal Information Processing Standard
GCM	Galois/Counter Mode
GEO	Geostationary Earth Orbit
GRBOOT	Gaisler Boot Software
GUI	Graphical User Interface
HAL	Hardware Abstraction Layer
HMAC	Hash-based Message Authentication Code
LEO	Low Earth Orbit
LMS	Leighton-Micali Signature
MAC	Message Authentication Code
ML-DSA	Module-Lattice-based Digital Signature Algorithm
NIST	National Institute of Standards and Technology
OBC	On-Board Computer
OTA	Over-the-Air
PKI	Public Key Infrastructure
PM	Processor Module
PoC	Proof of Concept
PQC	Post-Quantum Cryptography

Abbreviation	Definition
PUS	Packet Utilisation Standard
ROM	Read-Only Memory
RoT	Root of Trust
RSA	Rivest-Shamir-Adleman
SAVOIR	Space Avionics Open Interface Architecture
SDLS	Space Data Link Security Protocol
SEL	Single Event Latch-up
SEU	Single Event Upset
SHA	Secure Hash Algorithm
SLH-DSA	Stateless Hash-based Digital Signature Algorithm
SMP	Symmetric Multi-Processing
SRAM	Static Random-Access Memory
SUIT	Software Updates for Internet of Things
SWD	Serial Wire Debug
TC	Telecommand
TEE	Trusted Execution Environment
TID	Total Ionising Dose
TM	Telemetry
TRNG	True Random Number Generator
UART	Universal Asynchronous Receiver-Transmitter
UEFI	Unified Extensible Firmware Interface
WCET	Worst-Case Execution Time
XMSS	eXtended Merkle Signature Scheme

1

Introduction

1.1. Context - rise of COTS in space and the cybersecurity gap

From the beginning of human civilisation, the desire to understand the cosmos has been shaped not only by curiosity but also by technological limitations. Ancient philosophers such as Plato and Aristotle developed models of the universe based on observation and reasoning, constrained by the absence of precise instruments and experimental methods. For centuries, humanity's exploration of space remained theoretical, bounded by the tools and materials available at the time.

Even with the advent of modern science, early space exploration efforts were defined by severe technological constraints. The first spacecraft, such as Sputnik 1, were built using highly specialised, purpose-built components, often developed under strict governmental control. Systems were relatively simple by today's standards, with limited computational capabilities and minimal reliance on complex software. This inherent simplicity, combined with physical isolation and restricted access to hardware, resulted in a comparatively small attack surface.

In contrast, modern spacecraft development has undergone a fundamental transformation. Space missions increasingly rely on commercial off-the-shelf (COTS) components to reduce development costs and time to launch [14]. This results in a lower barrier to entry and a larger number of satellites being deployed for a wide range of applications. Organisations such as NASA and ESA, alongside private companies like SpaceX, now routinely integrate widely available processors, operating systems, and development frameworks into spacecraft architectures.

Despite being beneficial in many ways, this trend also introduces new cybersecurity risks [37, 55, 58]. Unlike the bespoke systems of earlier missions, modern spacecraft increasingly rely on hardware and software components that are widely documented, commercially distributed and, in some cases, publicly accessible. This accessibility significantly lowers the barrier for adversaries to analyse, replicate and potentially exploit system architecture.

While there have been security advancements such as the SDLS network protocol [43] and frameworks designed to better protect spacecraft systems [37, 13], other security measures remain less mature than their terrestrial counterparts [55, 20, 21], as reliability and fault tolerance have traditionally been prioritised. As discussed in [20], this disparity is driven by a range of technical and operational constraints. However, it also highlights the growing importance of spacecraft as potential single points of failure within critical infrastructure systems. Recognising this, ESA has identified cybersecurity as a strategic cross-cutting priority across all mission types [19, 15], encompassing areas from secure protocol standardisation to cryptography and security-by-design reference architectures. The establishment of a dedicated Cyber Security Operations Centre at ESOC in 2025 further underscores this institutional commitment [18].

In this context, ensuring the integrity of onboard software has become a critical requirement. Secure Boot mechanisms, which establish a hardware-rooted chain of trust during system initialisation, provide a means to prevent the execution of unauthorised or malicious code. While widely adopted in terrestrial

computing, their integration into spacecraft systems presents unique challenges due to constraints such as limited computational, space, and power resources.

1.2. Problem statement - why bootloaders are the critical attack surface

A bootloader is a small but critical software component responsible for initialising hardware and loading the operating system or main application software during startup [39, 6]. It acts as the first link in the software chain, bridging the gap between hardware and higher-level software. In embedded and spacecraft systems, bootloaders often handle additional tasks such as selecting firmware images, supporting remote updates, and providing recovery mechanisms in case of corruption [17, 2, 51].

As the bootloader executes before any other software, it holds a position of high trust. If an adversary manages to deliver malicious software to the system, the bootloader is the last line of defence that could prevent it from executing. A Secure Boot process mitigates this risk by cryptographically verifying each software component before execution, ensuring that only authenticated and untampered code is loaded.

While Secure Boot is widely adopted in terrestrial systems [54], it has yet to see broad application in space missions. This is mainly due to stringent reliability requirements, limited resources (computational, space, and power) and the high cost of qualifying new technology for space. It leaves a critical gap in protection against unauthorised code execution once a system has been compromised. There are recorded satellite attacks where the hackers managed to gain command-and-control (C2) access to the system [7]. The ENISA Space Threat Landscape report [12] identifies *firmware corruption* and *malicious code injection* as key cyber threats to satellite on-board systems. It notes that implanted malware can exploit software misconfiguration to manipulate the on-board controller, cause resource exhaustion, and ultimately lead to seizure of control, with the mission control centre potentially unable to identify the root cause until it is too late. Secure Boot could prevent adversaries from having persistent access to the satellite; therefore, investigating how it can be adapted for spacecraft offers an opportunity to significantly enhance the trustworthiness of space-borne systems, ensuring the integrity of the boot chain along with the final application software.

1.3. Research questions

The identified research gap motivates the central question of this thesis. It is formulated to address a practical design trade-off: how to strengthen protection against unauthorised code execution while preserving the reliability, performance and resource efficiency required by mission-critical spacecraft platforms.

How can Secure Boot be incorporated into space mission bootloaders in a way that enhances security without compromising the strict reliability and performance constraints of space systems?

This overarching question can be divided into the following sub-questions:

- How can Secure Boot be adapted using a secure-by-design approach compatible with space mission requirements?
- What is the minimal level of hardware support necessary for implementing Secure Boot in space-grade systems, given the high cost and limited flexibility of such hardware?
- Which existing terrestrial Secure Boot standards and frameworks can be applied to space environments?
- To what extent should post-quantum cryptographic algorithms be considered and do they necessitate dedicated hardware support?

1.4. Scope and contributions

This thesis makes the following contributions:

- **Threat model.** A lifecycle-based threat model for spacecraft bootloaders was developed, grounded in the ENISA Space Threat Landscape taxonomy [12] and focused on two primary attacker scenarios: physical access prior to launch and remote firmware injection after launch.
- **Secure Boot specification (SAVOIR-GS-002S extension).** A formal set of Secure Boot requirements was produced in the style and language of SAVOIR [51], designed to extend the existing SAVOIR-GS-002 Flight Computer Initialisation Sequence without replacing it. The specification covers a hardware Root of Trust, digital signature verification, classical and post-quantum support, rollback protection, and failure reporting.
- **Hardware trade-off analysis.** The hardware requirements for implementing the specification were analysed and a representative COTS microcontroller (STM32F439ZI) was selected for the proof of concept, demonstrating that the minimal hardware primitives needed — write-protected flash sectors and an optional hardware hash accelerator — are already present in widely available embedded processors.
- **Proof-of-concept implementation.** A working secure bootloader was implemented in C on the STM32F439ZI, realising all SEC requirements from the specification. The implementation includes A/B image partitioning, monotonic rollback protection via hardware-protected non-volatile storage, option-byte-enforced write protection, PUS-compatible boot reporting and an algorithm-agnostic cryptographic verification interface.
- **Performance benchmarks.** Signature verification latency and memory footprint were measured on the target hardware for four schemes — ECDSA, RSA, ML-DSA, and LMS — under both hardware-accelerated and software-only SHA, providing concrete data points for mission designers evaluating classical and post-quantum algorithm choices.

1.5. Report structure

Following this introduction, Chapter 2 presents background on bootloaders, secure boot, space standards and post-quantum cryptography. Chapter 3 then defines the threat model, including the two attacker scenarios considered in this work. Chapter 4 formulates the secure-by-design requirements and architectural decisions that extend existing ESA standards. Chapter 5 analyses the minimal hardware requirements needed to realise the design and motivates the platform selected for the proof of concept. Chapter 6 describes the implementation of this design on the selected hardware. Chapter 7 evaluates the proof of concept in terms of security properties and resource overhead. Chapter 8 discusses related work in secure bootloader development. Finally, Chapter 9 summarises the contributions and outlines directions for future work.

2

Background

2.1. Bootloaders

A bootloader is a set of software that is executed as the first thing following a reset/power-on. It is responsible for setting up the hardware, performing tests to ensure everything is in order, preparing the system to launch the main application, and finally transferring control to the main application [31, 2, 51, 57].

A system with a bootloader typically contains at least two coexisting software images: the bootloader itself and the main application [2, 51, 57]. On every startup, the bootloader first decides which image to load, checking conditions such as whether an update has been requested or whether a valid application image is present at all.

2.1.1. Bootloader Types

There are many types of bootloaders. They range from simple versions used for constrained embedded devices, where resources are limited and setting up the system might be straightforward, all the way to complex solutions used in consumer computers running operating systems such as Windows, Linux, or macOS.

Complex systems require a complex bootloader, which is why they are often split into multiple stages run one after another. The first stage — called BIOS or UEFI — is responsible for setting up the hardware and low-level peripheral drivers, while the second stage (e.g. GRUB) launches the operating system itself. Both stages may offer a comprehensive graphical user interface (GUI) for system configuration, but this complexity greatly enlarges the attack surface.

An intermediate case is a mobile operating system such as Android, which runs a full OS on constrained hardware using a single-stage bootloader [25]. Beyond loading the kernel, such a bootloader is also responsible for initialising the Trusted Execution Environment (TEE), binding the hardware root of trust, and verifying the integrity of all boot partitions before handing over control. When multiple firmware slots are present, it additionally selects the correct slot to boot. This tight integration of security responsibilities into the bootloader itself is a pattern applicable well beyond mobile devices.

Embedded computers are typically simpler, with a single-stage bootloader being common. While these systems can still run operating systems, they are usually much easier to set up, include fewer peripherals and support less functionality. Given that program space is scarce, the bootloader must be kept as small as possible to maximise the flash space available to the main application [2, 57].

2.1.2. A/B partitioning and OTA updates

One of the common functionalities of modern bootloaders is the ability to perform software updates while an older version is running [26, 36, 57]. By storing two copies of the application in separate partitions, often called slots A and B, one copy can be updated while the other continues to run. On the next boot, the bootloader selects the updated slot, with an automatic fallback to the previous version if

the new image fails to start correctly. This self-healing mechanism is particularly important in remote or unattended deployments where manual recovery is impossible. The A/B partitioning pattern is widely used in mobile devices and IoT firmware update frameworks [26, 36, 2] and is also applicable to space systems, where the ability to recover from a failed update is critical given the impossibility of physical intervention.

2.2. Secure boot fundamentals

Secure Boot — also referred to as verified boot, authenticated boot, or trusted boot depending on the implementation context — is a security mechanism that ensures only authorised, unmodified software executes on a device. The terms are broadly synonymous: the essential property is that before transferring control to the next software stage, the currently executing stage cryptographically verifies its successor's integrity and authenticity.

The mechanism operates as a chain: each verified stage in turn verifies the next one, extending a continuous chain of trust from an initial hardware root of trust all the way to the final application. Crucially, every link in this chain must be verified before execution; a single unverified transition breaks the security guarantee for all subsequent stages.

NIST SP 800-193 [42] frames platform firmware resiliency around three complementary principles: *Protection* (authenticated update mechanisms and write-protected storage preventing unauthorised modifications), *Detection* (integrity checks before code execution), and *Recovery* (restoring firmware to a known-good state after corruption is detected). Secure boot primarily addresses the Protection and Detection pillars, providing assurance that only authentic, unmodified firmware executes at each stage of the boot sequence.

2.2.1. Root of trust

In order to establish a secure chain of trust, each software component must verify the integrity and authenticity of the next component before transferring execution to it. While this is straightforward once trusted software is running, a different mechanism is required for the first stage bootloader, as it is the first software executed on the device. The verification cannot be performed by the bootloader itself, since an attacker could modify its initial instructions to bypass the verification process entirely.

NIST SP 800-193 [42] defines a *Root of Trust* (RoT) as “an element that forms the basis of providing one or more security-specific functions”, and requires that it “always behaves in the expected manner because its misbehaviour cannot be detected”. The standard further specifies that RoTs must either be immutable or be protected such that their integrity and authenticity are verified before any modification is applied. Building on a hardware RoT, a *Chain of Trust* (CoT) can be established by having each element cryptographically verify the next before transferring control, extending the trustworthy boundary all the way to the final application software.

This requirement means that trust in the initial boot stage must ultimately be established through hardware mechanisms. One approach is to use a dedicated hardware module that verifies the first-stage bootloader immediately before execution. Another approach is to place the initial boot code in read-only memory, ensuring that it cannot be modified after manufacturing. Once a trusted initial stage has been established, it can be used to verify all subsequent software components, thereby extending the chain of trust throughout the boot process.

2.2.2. Data verification

Establishing a root of trust alone is insufficient to secure the boot process. The trusted initial stage must also be capable of determining whether subsequent software components can be trusted before they are executed. This requires a mechanism for verifying both the integrity and authenticity of firmware images.

Data verification aims to ensure both the integrity and authenticity of received data [45, 5]. Integrity ensures that any modification of a firmware image can be detected, while authenticity provides assurance that the image originates from a trusted source and was generated by the entity it claims to represent. Together, these properties prevent an attacker from executing either modified firmware or firmware produced by an unauthorized party.

Both properties are essential for maintaining the security of software executing on a device [42]. In the context of Secure Boot, this means verifying that a firmware image has not been modified since it was produced and that it originates from an authorized manufacturer. This requires a cryptographic verification mechanism that enables the bootloader to determine whether an image can be trusted before transferring execution to it. The following subsections discuss common approaches for providing these integrity and authenticity guarantees.

Message Authentication Codes

One approach to providing integrity and authenticity guarantees is through message authentication codes (MACs), which are based on symmetric-key cryptography. A single shared secret key is used both to generate and verify the authentication tag. The HMAC construction [49] combines a cryptographic hash function with a shared secret, providing integrity through the hash function and authenticity through possession of the secret key.

Applied to secure boot, this requires that the bootloader and the manufacturer share the same secret key, which must therefore be stored on the device. If an adversary gains read access to the device memory and extracts the key, they can forge valid signatures for any firmware image — not only for that device but for any other device provisioned with the same key. While hardware read protection can partially mitigate this risk, symmetric schemes introduce a key-distribution and key-storage problem that makes them less suitable for secure boot than asymmetric alternatives.

Digital Signatures

Unlike symmetric MACs, digital signature schemes are based on asymmetric public-key cryptography. A *private key*, held exclusively by the firmware signer, is used to produce the signature. A corresponding *public key*, embedded in the bootloader at manufacturing time, is used to verify it. The two keys are mathematically linked, but the private key cannot be reconstructed from the public key alone [45].

In a secure boot context, the manufacturer retains the private key and ships each device with only the public key, stored in write-protected memory. Even if an adversary reads out the public key, they gain nothing useful: it cannot be used to sign new firmware images. The only way to produce a valid signature that the bootloader will accept is to possess the manufacturer's private key [45, 5].

Classical digital signature schemes are standardised in NIST FIPS 186 [45]. The foundational DSA algorithm has largely been superseded by elliptic-curve variants (ECDSA and EdDSA), which offer equivalent security at shorter key lengths. RSA-based schemes such as RSA-PSS are also standardised. Post-quantum digital signature schemes, which resist attacks by quantum computers, are discussed in Section 2.3.

2.2.3. Secure firmware updates

The mechanisms described above can be combined to build a secure firmware update process. By extending the A/B partitioning model from Subsection 2.1.2 with digital signature verification, updates can be delivered to the inactive slot and authenticated before the bootloader commits to booting from it. This ensures that a failed or tampered update is rejected and the device falls back to the previously running image.

Standardised approaches to this problem exist for constrained embedded systems. RFC 9019 [36] defines the Firmware Update Architecture for Internet of Things, specifying requirements for secure, authenticated updates including the use of asymmetric digital signatures, a signed manifest carrying image metadata, and support for post-quantum algorithms. RFC 9124 [35] complements it with a detailed Manifest Information Model, defining the structure and semantics of the manifest artefact. While these documents target IoT deployments, the underlying principles are broadly applicable to any embedded system where remote, unattended updates must be both reliable and trustworthy.

2.2.4. Rollback protection

Having a secure update mechanism not only allows installing new firmware, but also makes it possible to fall back to a previously running version if the new image fails to operate correctly. While this recovery capability is valuable, it also introduces an attack vector. If a critical vulnerability has been patched in

a new firmware release, an attacker who can force a downgrade effectively re-exposes the system to an already-known and exploitable flaw.

Rollback protection closes this gap by ensuring that the bootloader refuses to execute any firmware image whose version number falls below a stored minimum acceptable version. This minimum acts as a floor: images above it remain eligible for execution, including a fallback to a recent previous version, while anything below it is rejected unconditionally. The floor is typically maintained as a *monotonic counter* in a protected memory region that can only be incremented, never decremented. When a new firmware image is successfully booted and declared healthy, the counter is advanced to match the new version; all images bearing an older version field are subsequently rejected, regardless of their signature validity.

In the A/B partitioning model, rollback protection interacts directly with slot selection: the bootloader must not fall back to the inactive slot if doing so would violate the monotonic counter constraint. NIST SP 800-193 [42] formally identifies this as a *non-bypassability* requirement, noting that “unauthorized rollback could allow an attacker to restore a vulnerable firmware image, which in turn could allow the attacker to damage the device or inject malware” and that devices supporting rollback must include appropriate security controls to prevent exploitation by unauthorised parties.

2.3. Post-quantum cryptography

The secure boot mechanisms described in the previous section rely on digital signatures, which are built on classical public-key cryptography. The security of these schemes rests on mathematical problems that are believed to be hard for classical computers, but would be efficiently solvable by a sufficiently large quantum computer. For space systems, where a mission may span a decade or more and in-field algorithm replacement is impractical, this creates a concrete risk: a bootloader designed today may be protecting a spacecraft that is still operational when such a machine becomes available. Post-quantum cryptography (PQC) addresses this by providing signature schemes that remain secure even against adversaries equipped with large-scale quantum computers.

2.3.1. Motivation and timeline

Public-key cryptography underpins the security of digital services ranging from financial transactions and contract signing to firmware authentication. In the context of secure boot, a break in the underlying signature scheme would mean that an attacker could sign arbitrary firmware images and have them accepted by any device trusting the compromised key. Should the currently deployed schemes be broken, the consequences for digital infrastructure would be devastating [50, 28]. The principal threat is the development of a large-scale fault-tolerant quantum computer capable of running Shor’s algorithm, which renders classical public-key schemes such as RSA and ECDSA insecure. The state of quantum computer development is described in detail in [29].

Although no cryptographically relevant quantum computer (CRQC) exists today, its development is progressing rapidly. The 2023 Quantum Threat Timeline survey [50] polled 37 international experts from academia and industry: 17 estimated the probability of a CRQC appearing within a 10-year time frame to be above 5% and 10 indicated a likelihood of approximately 50% or more. These figures represent expert consensus that the quantum threat is not merely theoretical.

Two distinct threat scenarios motivate an early transition [50, 28]:

- **Store-now, decrypt-later:** Adversaries can record and store encrypted data today and decrypt it retroactively once a CRQC becomes available. Any data whose confidentiality must be maintained beyond the anticipated emergence of a CRQC is already at risk.
- **Long migration periods:** Complex systems such as spacecraft, public-key infrastructures (PKI) and devices with decade-long operational lifetimes require years to migrate. Even in the absence of ongoing attacks, there is a risk that the transition to quantum-safe cryptography may not be completed in time, endangering the authenticity of all future communication.

The urgency of the transition is underscored by a joint statement signed by cybersecurity agencies from 21 European nations [50], which calls on public administrations, critical infrastructure providers and industry to make the transition to PQC a top priority. Systems handling sensitive data should be

protected against store-now, decrypt-later attacks no later than the end of 2030 and detailed transition plans for PKI systems should be developed within the same time frame [50, 1]. The full migration away from classical public-key schemes is expected to be completed by 2035 [28].

2.4. The space environment and its constraints

Designing a secure bootloader for terrestrial embedded systems is already challenging; deploying one in space introduces additional constraints that fundamentally shape every architectural decision. Unlike ground-based platforms, a spacecraft cannot be physically accessed after launch, making repairs and manual recovery impossible. At the same time, the radiation environment degrades hardware in ways that have no terrestrial equivalent, and mission lifetimes measured in years or decades require cryptographic choices that remain secure and maintainable for the entire operational period. Additionally, every aspect of a space system — from hardware selection to software engineering practices — is governed by a body of international standards that constrain what is permissible and how it must be validated. Among these, the ECSS standards define engineering, quality and security requirements across the mission lifecycle, and the SAVOIR (Space Avionics Open Interface Architecture) initiative specifies open functional interfaces for space avionics, including the On-Board Computer. Understanding these constraints is essential context for the secure boot design developed in the following chapters.

2.4.1. Radiation tolerance

Space exposes electronics to radiation effects that have no meaningful terrestrial equivalent. The principal concerns are Total Ionising Dose (TID), which causes cumulative degradation of semiconductor properties over time, and single-event effects such as Single Event Upsets (SEUs), which cause transient bit-flips in memory or logic, and Single Event Latch-up (SEL), which can permanently destroy a component if left unchecked [32].

Processors and memories are classified according to how well they tolerate these effects. Radiation-hardened (rad-hard) devices are designed from the ground up to withstand high TID levels and are qualified to stringent standards (QML, ESCC). They are the choice for long-duration missions in harsh orbits such as Geostationary Earth Orbit (GEO) or deep space, where accumulated dose and mission lifetime both demand the highest reliability guarantees [32].

Radiation-tolerant (rad-tol) devices occupy a middle ground. They are typically derived from proven Commercial Off-The-Shelf (COTS) architectures but are enhanced with space-grade quality flows, latch-up immunity and improved TID characterisation. They are well suited to Low Earth Orbit (LEO) missions, where the radiation environment is less severe, operational lifetimes are shorter and cost is a primary driver [32]. The growing NewSpace sector, driven by private investment in LEO constellations, has significantly increased demand for such cost-effective solutions.

Plain COTS devices generally cannot meet space radiation requirements without additional mitigation such as shielding or latch-up detectors, which add mass and cost [32]. However, since rad-tolerant and rad-hard processors are typically derived from the same commercial architecture, software developed and tested on a COTS development board can be directly transferred to the space-grade variant, making COTS hardware a practical platform for BSW development and validation.

This extends to storage as well. Space-grade NOR flash memory supports a limited number of write-erase cycles, and radiation-hardened variants impose stricter constraints than commercial equivalents [27]. This limits how many firmware updates can be applied over a mission lifetime and must be accounted for in the update strategy. The more expensive and less dense ferroelectric RAM (F-RAM), also available in radiation-hardened form, offers effectively unlimited write endurance and is used in space applications precisely to avoid this constraint [27].

2.4.2. Resource constraints - compute, power, memory

Space-grade processors are generally slower than their terrestrial counterparts, a consequence of the conservative fabrication processes required for radiation hardness. Combined with strict power budgets driven by the limited energy available from solar panels and batteries, this means that computationally expensive operations — such as verifying a large digital signature — must be carefully considered in the context of the overall boot-time budget. During a system restart, both processing modules need

to return to an operational state as quickly as possible, since the redundant copy provides the only fail-safe if the active one encounters a problem. Boot time is therefore a design constraint that directly interacts with the choice of cryptographic algorithms.

2.4.3. SAVOIR - Generic OBC specification

The SAVOIR Generic OBC Specification [51] (SAVOIR-GS-001) specifies the functional requirements for the On-Board Computer used in space missions, covering telecommand and telemetry handling, on-board time management, data storage, fault detection and reconfiguration, and processing.

The architecture mandates two Processor Modules (PMs) operating in warm or cold redundancy, following an active/inactive scheme [51]. The active PM executes the Application Software (ASW) and has full access to avionics interfaces; the inactive PM operates in a limited functionality mode used for testing, reprogramming, or fast switchover. Ground has no direct communication path to the inactive PM: all commands are relayed through the active PM via the inter-PM link, and the active PM is responsible for reading and writing the software storage memory of the inactive PM to perform ASW updates.

Memory organisation is explicitly addressed by the specification [51]. Each PM must have a write-protected boot memory for its Boot Software, ensuring that the first code executed after reset cannot be overwritten in normal operation. Software images reside in a separate non-volatile storage area. The number of times this storage can be reprogrammed over the OBC lifetime is bounded by the endurance of the underlying non-volatile memory technology — the specification notes a typical value of 1000 cycles, which may be reduced to 200 for long-life missions.

2.4.4. SAVOIR - Flight Computer Initialisation Sequence

The SAVOIR Flight Computer Initialisation Sequence specification [51] (SAVOIR-GS-002) defines the required behaviour of the Boot Software (Boot SW) for OBC and Payload Computers. The Boot SW is the first code executed on every processor reset or power-on. It is intentionally agnostic to the unit's nominal function, meaning the same boot process applies equally to an OBC, a payload computer, or an instrument processor. The Boot SW is stored in dedicated read-only memory — either a PROM or a write-protected EEPROM — making it the immutable starting point from which all software recovery begins.

The specification defines three operating modes: Nominal Sequence, Standby, and Monitor. The Nominal Sequence is the primary boot path and is the first code executed after reset. It follows a fixed seven-step procedure:

1. Perform processor module initialisations
2. Perform processor module Self-Tests
3. Select the active Application Software (ASW) image
4. Test the integrity of the selected ASW image in Application storage memory
5. Copy the selected ASW image from Application storage memory to working memory
6. Test the integrity of the copied ASW image in working memory
7. Execute the ASW from working memory

At least two ASW images must be maintained in Application storage memory to allow fallback to a previously known-good version. On multi-core processors, only one core executes the Boot SW; secondary cores remain disabled throughout the boot sequence and are initialised by the ASW once it takes control. The Boot SW produces a Boot Report after the Self-Tests, recording their outcome in Safeguard Memory so that the result is available for ground analysis even after subsequent resets.

The Boot SW is deliberately kept simple in terms of fault handling: it must continue the boot sequence even if Self-Tests fail, leaving FDIR to the ASW. If meeting the worst-case execution time (WCET) budget for a given reconfiguration scenario requires skipping steps, a Fast Boot Path allows bypassing Self-Tests or integrity checks; this decision is configured before the reset by the system manager.

Standby mode is used by an inactive processor module to remain reachable for maintenance while the active PM holds operational control, as described in Section 2.4.3. Monitor mode is a ground-only debug facility that must be disabled before flight.

Of particular relevance to this thesis is Section 5.8 of SAVOIR-GS-002, which explicitly marks security as “Not applicable.” The integrity checks in steps 4 and 6 compute a CRC or checksum over the ASW image and compare it against a stored reference value, providing protection against random bit corruption but no protection against deliberate modification. An attacker with write access to Application storage memory could update both the image and its stored checksum, rendering the integrity check ineffective. The SAVOIR specification imposes no authentication or cryptographic verification requirements on the boot process, leaving secure boot entirely outside the standardised interface.

2.4.5. ECSS Standards

ECSS-E-ST-40C: Software Engineering [8] defines the general software engineering requirements for space systems. Its purpose is to ensure that software developed for space missions follows a structured, well-documented and verifiable engineering process throughout the entire lifecycle.

The standard covers activities such as requirements definition, architectural design, implementation, verification, validation, configuration management and maintenance. It is intended to improve software reliability, maintainability and correctness in the context of highly critical space systems.

ECSS-Q-ST-80C: Software Product Assurance [11] specifies software product assurance requirements for space projects. Its main objective is to ensure that software products achieve the required levels of quality, dependability and safety.

The standard introduces requirements related to software planning, risk management, verification independence, problem reporting and quality metrics. It also defines how software assurance activities should be tailored based on software criticality and project constraints.

ECSS-E-ST-80C: Security in Space Systems [10] addresses security aspects across the lifecycle of space systems. Its purpose is to ensure that security is considered systematically during system design, development, operation and disposal.

The standard covers topics such as threat analysis, secure design principles, protection of sensitive assets and implementation of security mechanisms. For software systems, it emphasises the establishment of trust, protection against unauthorised modification and mitigation of cyber threats.

ECSS-E-ST-70-41C: Telemetry and Telecommand Packet Utilization [9] defines the Packet Utilization Standard (PUS). PUS specifies the application-layer interface between ground systems and spacecraft by standardising how telecommand (TC) and telemetry (TM) packets are structured and interpreted at the mission application level. It builds on the CCSDS Space Packet Protocol and introduces a service-oriented model in which related on-board functions are grouped into numbered service types (ST[xx]). Communication follows a request-report pattern: a ground station issues a TC request, and the onboard software responds with one or more TM reports. PUS defines 23 standard service types covering functions such as telecommand verification (ST[01]), housekeeping reporting (ST[03]), event reporting (ST[05]), on-board scheduling (ST[11]) and memory management (ST[06]), among others.

Service 6 (ST[06]) is the PUS service for on-board memory management [9]. It provides ground operators with a standardised interface for reading, writing and verifying spacecraft memory contents. It is organised into four subservices. The *raw data* subservice operates on untyped byte sequences using absolute addresses, supporting load (TC[6,2], TC[6,11]), dump (TC[6,5]) and checksum verification (TC[6,9]) operations. The *structured data* subservice uses a base-plus-offset addressing scheme for named, typed memory objects, with equivalent load (TC[6,1]), dump (TC[6,3]) and check (TC[6,7]) operations. The *common* subservice allows aborting all in-progress dumps (TC[6,12]). The *memory configuration* subservice controls hardware write protection (TC[6,15] enable, TC[6,16] disable) and memory scrubbing (TC[6,13] enable, TC[6,14] disable).

ST[06] is directly relevant to the secure boot context of this thesis. The memory load and write-protection telecommands provide the standard mechanism by which ground operators upload and protect ASW images — the same images whose integrity the Boot Software must verify before exe-

cution. If these telecommands are not properly authenticated, they represent a direct attack vector against the boot chain, as discussed in Section 9.3.1.

2.4.6. Cryptographic recommendations for space

Space systems present unique challenges for cryptographic design. Long mission lifetimes, constrained hardware resources and the absence of on-site maintenance mean that cryptographic choices made at design time must remain secure and maintainable for a decade or more. Several national and international bodies publish cryptographic recommendations relevant to such systems, including the German BSI [28], the French ANSSI [1] and the space-specific CCSDS Cryptographic Algorithms Recommended Standard [5]. While each authority has its own scope and focus, they broadly converge on a common set of algorithm families and migration timelines driven by the threat of quantum computers. This convergence is further reinforced by a joint statement from cybersecurity agencies across 21 European states [50], which calls for urgent and coordinated transition to PQC.

Key lengths for classical signature algorithms. Table 2.1 summarises the minimum key lengths recommended for classical digital signature mechanisms. These correspond to a security level of at least 120 bits against classical adversaries [28]. CCSDS [5] specifies stricter requirements for new space implementations: an RSA key length of at least 4096 bits.

Table 2.1: Recommended minimum key lengths for classical digital signature mechanisms.

Authority	RSA-PSS (modulus)	ECDSA (order q)	Hash function output
BSI [28]	3000 bits	250 bits	SHA-256 minimum
CCSDS [5]	4096 bits	permitted	SHA-256 minimum

Migration timeline. Classical signature schemes will not remain recommended indefinitely. Table 2.2 summarises the migration deadlines for digital signatures as defined by BSI [28], the EU PQC roadmap [50] and ANSSI [1], which are broadly aligned with NIST guidance.

Table 2.2: Migration deadlines for classical digital signature schemes.

Deadline	Requirement
2024–2025	Products with firmware that must remain trustworthy beyond 2030 should begin hybrid signature deployment immediately [1].
End of 2030	Systems handling sensitive long-lived data must be protected against store-now, decrypt-later attacks. Detailed transition plans for PKI systems and long-lived devices should also be completed within this timeframe [50].
End of 2035	Classical signature schemes (RSA, ECDSA) recommended only until this date; full transition to quantum-safe or hybrid signatures required [28].

Recommended quantum-safe signature algorithms. Table 2.3 summarises the quantum-safe signature schemes recommended for data authentication and secure boot by the authorities surveyed. All are standardised by NIST. A notable remark from BSI is that stateful hash-based schemes (LMS/HSS, XMSS) are explicitly endorsed for *“the signing of software and firmware updates”* as a scenario in which error-free state management can be guaranteed [28].

CCSDS space-specific cryptographic recommendations. CCSDS 352.0-B-2 [5] is the Consultative Committee for Space Data Systems recommended standard for cryptographic algorithms, specifically targeted at civilian space missions. For digital signatures, it mandates the Digital Signature Standard (DSS, FIPS 186-3) [5]; RSA at 4096 bits is required for new implementations, while ECDSA is also permitted. The hash function used with these signatures must be SHA-256 or stronger; SHA-1 is explicitly prohibited. Note that CCSDS 352.0-B-2 predates the NIST PQC standardisation effort;

Table 2.3: Recommended quantum-safe signature schemes for firmware authentication.

Scheme	Standard	Notes
ML-DSA	FIPS 204 [46]	Lattice-based (MLWE/MSIS). Hedged variant; parameter sets 65 and 87 (NIST Categories 3 and 5). Pure-message variant preferred over pre-hash variant [28]. ANSSI recommends avoiding parameter modification and prefers levels 5 or 3 [1].
SLH-DSA	FIPS 205 [48]	Stateless hash-based. Hedged variant; Categories 3 and 5. Pure-message variant preferred over pre-hash variant [28]. Security based on preimage resistance only — most conservative assumption. Hybridisation optional per ANSSI and BSI [1, 28].
LMS/HSS	NIST SP 800-208 [47]	Stateful hash-based. All parameter sets recommended. Requires monotonic state counter; endorsed for firmware signing. Hybridisation optional [1].
XMSS	NIST SP 800-208 [47]	Stateful hash-based. All parameter sets recommended. Same state-management caveat as LMS/HSS. Hybridisation optional [1].

post-quantum algorithm recommendations for space are expected to follow in a future revision, but the general requirements already align with the broader migration guidance discussed above.

ANSSI-specific guidance. ANSSI [1] places particular emphasis on hybridisation: for any final product that includes post-quantum mitigation, hybridisation is *mandatory* unless the quantum mitigation relies solely on hash-based signatures (LMS, XMSS, SLH-DSA), for which it is optional. This position is shared by BSI [28]. ANSSI also advises preferring the highest available NIST security level — level 5 (equivalent to AES-256 classical security) or at minimum level 3 (equivalent to AES-192) — for all deployed PQC signature schemes and discourages any modification of standardised parameter sets [1].

Hybridisation combiners. Current guidance recommends deploying quantum-safe signature schemes in a *hybrid* configuration alongside a classical scheme (e.g. ECDSA + ML-DSA), accepting the combined signature only if both components are valid. This combiner is proved secure under the standard EUF-CMA model: security is retained as long as at least one constituent scheme remains unbroken [1]. This hedges against implementation weaknesses in the newer PQC algorithms while the classical component continues to provide well-understood protection [28].

3

Threat Model

3.1. Threat modelling methodology (ENISA lifecycle approach)

The threat model in this work follows a lifecycle-based approach for space as defined by ENISA [12], while focusing on threats that are directly relevant to the design and evaluation of a bootloader implementing secure boot. Rather than analysing each lifecycle phase individually, the model concentrates on two attacker scenarios that pose the greatest risk to the integrity of the boot process.

The ENISA Space Threat Landscape report [12] defines a comprehensive threat taxonomy for commercial satellite systems, covering threat categories from nefarious activity and physical attacks to legacy infrastructure risks. Of these, two threat categories are directly relevant to the bootloader: *Firmware corruption* — the overwriting or corrupting of firmware memory to render the system inoperable or to inject malicious functionality — and *Sabotage through hardware/software*, which covers the connection of unauthorised hardware or software (such as a USB storage device) to implant malicious code during the assembly, integration, or transport phase. The ENISA taxonomy also identifies *Malicious code/software/activity*: *Malicious injection*, explicitly listing “injection of malicious software (e.g. rootkits, bootkits, backdoors) into space operations control systems” and “on-orbit software updates, upgrades, patches, or direct memory writes” as concrete attack vectors. The two attacker scenarios below map directly onto these ENISA threat categories, grounding the threat model in a well-established, space-sector-specific framework.

3.2. Attacker scenarios

3.2.1. Physical access prior to launch

The first scenario considers an attacker who gains physical access to the system after it has left the manufacturing facility but before launch. This period typically includes assembly, integration and pre-launch testing, during which development or intermediate software versions may be present and hardware debug interfaces may be accessible.

An attacker with physical access may attempt to replace or modify bootloader code, inject unauthorised firmware images, alter boot configuration parameters, or extract cryptographic material used for image authentication. If successful, such modifications could allow the execution of malicious software that persists after launch. The ENISA Space Threat Landscape [12] identifies this concretely in its risk assessment: Scenario 2 describes an attacker gaining physical access to a satellite’s assembly line or transport container and implanting malicious code via an IO interface (e.g. a USB port), exploiting inadequate hardening procedures after assembly. ENISA notes that once in orbit such malware can execute remote exploits against the on-board controller and software, causing resource exhaustion and ultimately seizure of control — and that the ground team may be unable to identify the root cause before it is too late. Given that hardware access is no longer possible once the system is in orbit, preventing the execution of unauthorised code introduced during this phase is a primary security objective of the secure bootloader.

3.2.2. Remote access after launch

The second scenario considers a remote attacker who gains privileged access to the system after it has left the ground, during in-orbit testing or operational phases. In this scenario, the attacker cannot physically modify the hardware but may attempt to upload modified firmware images, exploit software vulnerabilities, or abuse update and recovery mechanisms to gain control of the system.

From the perspective of the secure bootloader, the main threat is the execution of unauthorised or modified firmware delivered through remote interfaces. The ENISA taxonomy [12] captures this under *Malicious code/software/activity: Malicious injection*, noting that malicious code “could also be injected into software updates, affecting for example on-orbit software updates, upgrades, patches, or direct memory writes”. The ENISA report further calls out *Seizure of control: Satellite bus* — adversaries exploiting existing vulnerabilities to execute malicious commands or lock out legitimate operators — as a high-impact consequence of successful remote attacks. Digital signature verification during the boot process is therefore assumed to be a critical control, ensuring that only authenticated and integrity-protected images can be executed, even if an attacker gains access to communication channels or higher-level software components.

By focusing on these two attacker scenarios, the threat model emphasises the role of secure boot as a foundational trust mechanism. In particular, it addresses the risk of persistent compromise introduced through physical access prior to launch and the risk of unauthorised firmware execution resulting from remote attacks after launch, both of which are central to the security guarantees provided by a signature-based boot process.

3.3. Scope of protection

Secure boot severs the link between *gaining access* and *gaining persistent control*. The real-world scenario motivating this work is one in which an attacker — whether through a pre-launch physical intrusion or a post-launch software exploit or telecommand abuse — manages to deliver a modified firmware image to the system. Without secure boot, that image would execute on the next reboot, giving the attacker persistent code execution that survives power cycles and is invisible to the ground team. With secure boot in place, the Boot SW rejects any image that does not carry a valid signature from the holder of the private signing key. The system returns to a clean, verified state after every reboot, and persistent malicious code execution is impossible without access to the private key. This is the central security guarantee this work aims to provide.

It is equally important to be precise about what this guarantee does not cover, since overstating the protection offered by secure boot would give a false sense of security. The following limitations are explicit non-goals of this threat model and the design that follows from it.

Availability. Secure boot prevents malicious code execution — it cannot recover missing or corrupted images. An attacker with privileged access at the application level or via authenticated telecommands can corrupt or erase flash partitions, issue destructive PUS memory-write commands, or exhaust system resources. If no valid, signature-verified image remains, the system becomes unbootable regardless of the integrity of the Boot SW itself. Protecting against such availability attacks requires complementary controls at the command-handling and ground-segment layers, which are out of scope for this work.

Private key compromise. The entire security guarantee is conditional on the private signing key remaining secret. If the signing infrastructure is breached and the key is extracted or misused, an attacker can produce validly-signed malicious images that pass all Boot SW verification checks. This work assumes the key management and signing process are secured by out-of-band means — hardware security modules, access controls, and audit procedures — but does not specify or evaluate those mechanisms.

Runtime attacks after successful boot. The trust guarantee established by secure boot ends at the moment execution is transferred to the Application Software. Memory corruption exploits, logic vulnerabilities, or privilege escalation in the running ASW are entirely beyond the boot-time trust boundary. Mitigating such attacks requires runtime protection mechanisms — memory protection units, watchdog supervision, anomaly detection — which are orthogonal to the boot process and out of scope here.

Communication channel attacks. Secure boot does not protect the channel through which firmware images or telecommands are delivered. An attacker who can jam, spoof, or replay the RF uplink may prevent valid software updates from reaching the system, force repeated reboots, or corrupt in-transit image data. Channel confidentiality, integrity, and availability are the responsibility of a separate communications security layer and are not addressed by this work.

4

Design

4.1. Design approach - secure by design, SAVOIR-compatible

The main contribution of this work is the proposed Secure Boot SW design, and therefore particular care was taken to consider all relevant information. Following a review of existing standards, research, and recommendations from the space segment, a specification document was developed that could serve as an extension to the existing SAVOIR framework. The objective was to achieve a secure-by-design solution, with security treated as a primary design consideration rather than an afterthought. This section describes how the findings and recommendations presented in Chapter 2 were incorporated into the proposed design.

4.1.1. Existing material

The specification draws on two preceding chapters. Chapter 2 established the technical foundations: how space BSW is structured, what the SAVOIR FCIS framework already mandates, how A/B partitioning and over-the-air updates work, and what the cryptographic literature recommends for long-duration systems including the transition to post-quantum algorithms. Chapter 3 then defined the attacker scenarios the design must defend against: a physical adversary with pre-launch hardware access, and a remote adversary operating through the communications and update channels after deployment.

Together these inputs shaped three core design decisions. First, SAVOIR already requires the BSW to reside in write-protected Boot Memory and defines integrity checks on ASW images; the gap is that those checks are CRC-based and provide no authenticity guarantee against a deliberate adversary. The specification closes this gap by adding cryptographic signature verification on top of the existing integrity checks, leaving the SAVOIR structure otherwise intact. Second, A/B partitioning is already present in space BSW; the specification extends it with rollback protection to prevent an attacker from exploiting the update channel to reinstall a vulnerable older version. Third, the background survey highlighted that missions with lifetimes of ten years or more face a realistic risk that classical signature algorithms may be weakened or broken before end of missions due to advances in quantum computing. The specification therefore requires that the digital signature algorithm provides resistance against quantum computational attacks.

4.2. Secure boot specification (SAVOIR-GS-002S extension)

In order to make the requirements easy to integrate, the same language and structure as existing SAVOIR documents were used. Additionally, standardised keywords are employed to indicate requirement levels, according to RFC 2119 [4]. Based on the research, all of SAVOIR-GS-002 can be kept as-is, with additional requirements added on top. The one exception is **SAVOIR.BOOTSW.BEF.22**, which permits the Fast Boot Path to execute a subset of the nominal sequence steps. Because digital signature verification (**SEC.03**) is a newly introduced step, BEF.22 must be constrained so that signature verification cannot be placed in the skippable subset; this constraint is captured as requirement **SAVOIR.BOOTSW.BEF.22S** in Appendix A, which lists the full formal requirements. This section

explains the reasoning behind them in more detail.

4.2.1. Establishing a hardware root of trust

The entire Secure Boot chain depends on a fundamental question: what is the first piece of software that can be trusted, and on what basis? As the Boot SW is the first software executed during system startup, it cannot verify its own integrity through software mechanisms. An attacker capable of modifying the Boot SW could alter its initial instructions and bypass all subsequent verification steps. Consequently, the root of trust cannot reside in software and must instead be established through hardware.

There are two hardware mechanisms that address this. First, the BSW itself must reside in a memory region that is physically immutable at runtime (a write-protected or one-time-programmable flash sector) so that no software, including compromised ASW, can overwrite it. Second, the public key used to verify the ASW images must be stored in an immutable region as well. Although an adversary who reads the public key gains no useful information, one who can replace it could substitute their own key and sign arbitrary images. Immutable storage disables this attack path entirely. If the immutable storage is limited, a hash of the public key could be stored instead with the public key being included as part of the ASW images.

This approach is consistent with NIST SP 800-193 [42], which requires that Roots of Trust “shall either be immutable or protected using mechanisms which ensure all RoTs and CoTs remain in a state of integrity” (§4.1.1) and that “the write protection of field non-upgradable memory shall not be modifiable” (§4.2.2). The standard further specifies that RoT functions “shall be resistant to any tampering attempted by software running under, or as part of, the operating system on the host processor” — a property achieved here by completing all BSW verification before any ASW code is executed.

These two constraints together — immutable Boot SW and immutable public key — constitute the hardware Root of Trust from which all subsequent verification steps derive their trustworthiness (**SEC.01-02**). All higher-level security guarantees in this specification are contingent on this foundation holding as well as the private key not becoming compromised.

4.2.2. Authenticated execution

With a trusted starting point established, the specification allows for the trust to be forwarded to the ASW using digital signature verification. Before transferring execution to any ASW image, the Boot SW must verify its digital signature across all image sections including the metadata. A valid signature provides two guarantees simultaneously: integrity and authenticity, ensuring the image has not been modified after signing and that it was signed by a trusted party.

NIST SP 800-193 [42] formalises this as an *Authenticated Update Mechanism* (§4.2.1.1), requiring that “firmware update images shall be signed using an approved digital signature algorithm” and that “the digital signature of a new or recovery firmware update image shall be verified by an RTU or a CTU prior to the non-volatile storage completion of the update process”. The specification also mandates *non-bypassability* (§4.2.1.3): there must be no development or diagnostic interface, architectural feature, or low-level vulnerability that allows an attacker to modify firmware outside the authenticated update mechanism.

The consequence of a failed verification is deliberately strict: the Boot SW must reject the image and prevent execution entirely (**SEC.03-04**). No fallback exists that permits unsigned or unverified code to run, since any exception would present a path for adversaries to take advantage of. The behaviour on failure is instead handled separately through the rollback and reporting mechanisms described below.

4.2.3. Post-quantum cryptographic support

A bootloader deployed on a long-duration mission may remain in operation well beyond the point at which its original cryptographic algorithms are considered secure. As discussed in Section 2.3, classical algorithms such as RSA or ECDSA are vulnerable to sufficiently powerful quantum computers, and the transition to post-quantum cryptography is expected to be complete by 2035. For missions with lifetimes of ten years or more, there is therefore a realistic risk that the signature algorithm used to verify the ASW image at boot time may be weakened or broken before end of mission.

The specification addresses this by requiring that the digital signature algorithm used for ASW image

verification provides resistance against quantum computational attacks (**SEC.05**). Algorithm selection remains mission-configurable, and a single algorithm is sufficient provided it offers resistance against both classical and quantum adversaries — hash-based schemes such as LMS/HSS satisfy this property by design.

4.2.4. Rollback protection

A/B partitioning, as described in Subsection 2.1.2, is already included in space BSW and provides recovery from failed updates. However, it introduces a risk where an attacker with access to the update mechanism could deliberately install an older, more vulnerable version of the ASW, undoing a security patch. Rollback protection closes this attack path. NIST SP 800-193 [42] explicitly identifies this threat in §4.2.1.3, stating that “unauthorized rollback could allow an attacker to restore a vulnerable firmware image, which in turn could allow the attacker to damage the device or inject malware”, and requires that authenticated update mechanisms “be capable of preventing unauthorized updates of the device firmware to an earlier authentic version that has a security weakness”.

Each ASW image carries a strictly positive integer version identifier. The Boot SW maintains a monotonic counter in protected non-volatile memory (which can be updated by the BSW only), recording the highest version that has been executed. Execution is then permitted only for images whose version falls within the inclusive range $[N, N - X]$, where N is the counter value and X is the mission-configured rollback window size (**SEC.06**).

A window parameter allows for greater flexibility based on the mission-specific needs. A window of zero would provide the strongest security but would eliminate any autonomous recovery from a defective update, requiring ground intervention for every rollback. A window of one permits recovery to the immediately preceding version while still preventing an attacker from reaching arbitrarily old and vulnerable images. A larger window size might also be applicable in specific cases. Storing the counter in protected non-volatile memory ensures it persists across resets and cannot be manipulated by the ASW, which is essential given that a compromised ASW must not be able to lower its own minimum version floor.

4.2.5. Failure visibility

In terrestrial systems, a boot failure is immediately visible. On a spacecraft, the only diagnostic channel is telemetry and the window for ground intervention might be limited or delayed. A silent failure could be indistinguishable from a hardware fault, leading to incorrect corrective action.

For this reason, the design requires the Boot SW to report secure boot failures and their specific causes through the Boot Report or telemetry mechanisms already defined in SAVOIR (**SEC.07**). Reportable causes include signature verification failure, use of an unrecognised or revoked key and version window violations. This gives ground operators the information needed to distinguish a genuine attack from a corrupted image or a configuration error and to take corrective action such as uploading a replacement image. This requirement is aligned with NIST SP 800-193 [42] §3.6.5, which prescribes event logging for firmware security events for forensic analysis and audit purposes, and with §4.3.1, which requires that detection mechanisms “should be capable of creating notifications of corruption” and “should be capable of logging events when corruption is detected”.

4.3. Key design trade-offs and discarded options

Not every security/safety mechanism considered during the design process made it into the final specification. Some were discarded because they conflicted with other requirements, others because their added complexity was not justified by the security benefit they provided.

One notable candidate was the concept of a golden image [16] — a read-only, pre-launch copy of the ASW stored in a dedicated partition and used as a last resort if all other partitions are corrupted or unbootable. This idea improves the systems safety as it provides a recovery path that does not depend on the update mechanism functioning correctly. However, it goes directly against rollback protection. A golden image is by definition fixed at the version present before launch, meaning that as the mission progresses and newer ASW versions are deployed, the golden image will inevitably fall outside the permitted rollback window. Exempting it from the version check would require a special

case path in the Boot SW, which introduces both complexity and a potential bypass vector, allowing an attacker who can trigger the golden image path to effectively force execution of an old, potentially more vulnerable version of the ASW. Making the golden image sector read-only would prevent the adversary from replacing its contents, but would not prevent them from exploiting the existing vulnerabilities in that version. Given that the rollback window provides a controlled recovery mechanism and the ground operators have the ability to upload new images via the Standby mode, the golden image concept was discarded in favour of simpler and more consistent design.

4.4. How does this address the threat model?

The two attacker scenarios defined in Chapter 3 place different demands on the design and it is worth examining how the specification responds to each.

For a physical attacker, the critical window is the period between manufacturing and launch, during which the hardware may be accessible and debug interfaces may still be enabled. The immutable storage requirements (**SEC.01-02**) are the primary defences here. Since the BSW and its public key(s) reside in write-protected memory, a physical attacker cannot modify the verification logic or substitute a different key without triggering a full device erase, which would be immediately apparent. In practice, the correct pre-launch procedure is to reset the ASW partitions and upload the mission images immediately before the system is finalised, at which point debug access should be revoked. Since the BSW and public key(s) are immutable by that point, no modification made during earlier integration phases can survive into flight.

For a remote attacker operating after launch, the threat surface shifts to the update and communication channels. Here the signature verification requirements (**SEC.03-04**) form the central control: regardless of how an attacker delivers a modified firmware image (whether by exploiting a software vulnerability, intercepting a telecommand or abusing the update mechanism), the Boot SW will refuse to execute it unless it carries a valid signature from the holder of the private key. Since the private key never leaves the ground segment, this guarantee holds as long as the key itself is not compromised. The rollback protection (**SEC.06**) adds a further constraint, ensuring that even a validly signed but deliberately outdated image cannot be used to reintroduce previously patched vulnerabilities. Together, these mechanisms mean that an attacker without access to the private key has no viable path to persistent code execution on the system, regardless of what access they have achieved at the application level.

5

PoC Scope and Hardware Selection

5.1. From specification to proof of concept

The design specification in Chapter 4 draws on two sources: the existing SAVOIR-GS-002 Flight Computer Initialisation Sequence, which defines the baseline behaviour expected of any space-grade bootloader and the new SEC requirements introduced in this work, which add the security guarantees that SAVOIR FCIS currently lacks. Together these form a complete picture of what a secure spacecraft bootloader should do. The purpose of the proof of concept is not to implement this picture in full, but to demonstrate that the security-critical subset is feasible on representative hardware.

The PoC requirements derived below therefore reflect two deliberate choices. First, all seven SEC requirements are carried forward without exception. These are the core contributions of this work and a PoC that omitted any of them would fail to validate the central claim that secure boot can be integrated into a space-grade bootloader without compromising the essential behaviour. Second, a subset of SAVOIR-GS-002 requirements is kept — specifically those without which the resulting system would not be recognisable as a space bootloader at all. Requirements covering the nominal boot sequence, standby mode, memory architecture and boot reporting fall into this category. Requirements covering multi-processor operation, monitor mode, fast boot paths and other non-essential bootloader functionality were put out of scope as they do not affect the desired security properties and would add substantial complexity without advancing the research objectives.

The PoC requirements derived from SAVOIR-GS-002 are listed in Table 5.1. The security requirements driving this PoC are specified formally in Appendix A as **SEC.01–SEC.07**. All seven requirements are carried forward into the PoC without exception.

PoC Req. ID	Derived PoC Requirement	Source Requirements
<i>Group 1 - Functional Requirements (SAVOIR-derived)</i>		
POC-FUNC-INIT	The BSW shall execute upon system reset, initialise and self-test the hardware, reset volatile memory to a known state after tests and set the processor in a known configuration before proceeding to the Nominal Sequence.	BEF.10 BIN.215 BIN.220 BIN.240 BIN.250 BTE.270 BTE.280 BTE.290 BAA.340
POC-FUNC-NOMINAL	The BSW shall execute the full nominal sequence: select the active ASW image based on ground-configurable data, test its integrity before and after copying it to working memory and transfer execution to it.	BEF.20 BEF.25 BEF.27 BTE.300 BTE.310 BTE.330
POC-FUNC-STANDBY	The BSW shall implement Standby mode, triggered by ground-controlled conditions checked within the Nominal Sequence, supporting memory inspection and ASW image updates from ground. The Standby function shall be entered regardless of self-test results and shall access only resources local to the processor module.	BEF.05 BEF.70 BEF.75 BEF.80 BEF.90 BEF.95 BEF.97
POC-FUNC-REPORT	The BSW shall record the result of each test step progressively in a Boot Report, stored in predefined areas of working memory and protected non-volatile storage and accessible to both the ASW and Ground.	BAA.370 BAA.390 BMM.130 BMM.140
<i>Group 2 - Memory Architecture Requirements (SAVOIR-derived)</i>		
POC-MEM-LAYOUT	The BSW code and read-only initialisation data shall be stored in Boot Memory. All remaining BSW code, integrity test logic and Standby functions shall be stored in designated flash regions. The BSW shall not depend on the actual parameter values of any ASW image and all memory regions used by the BSW shall be predefined.	BMM.100 BMM.110 BMM.120 BMM.170
<i>Group 3 - Design and Performance Constraints</i>		
POC-IMPL-DESIGN	The BSW shall execute sequentially without dependency on an operating system or run-time environment and shall use the minimum processor resources necessary to fulfil its requirements.	IMP.460 IMP.470
POC-PERF-WCET	The BSW worst-case execution time shall be measured and recorded for each supported signature algorithm, providing a basis for evaluating compatibility with mission reconfiguration time budgets.	BPF.450

Table 5.1: Secure Boot PoC Requirement Flow-Down and Traceability

5.2. Hardware requirements for space-grade secure boot

With the PoC requirements established, the next question is which of them impose constraints on the hardware platform itself. Several requirements are purely software concerns — SEC.05 (post-quantum support) and SEC.07 (failure reporting) and POC-FUNC-REPORT (boot reporting) — and can

be satisfied regardless of the underlying platform. The remaining requirements place specific demands on hardware that must be met before any software can address them; these are examined below.

5.2.1. Immutable storage for RoT and public key

The most fundamental hardware constraint comes from SEC.01 and SEC.02. Immutable storage for both the BSW and the public key requires a platform that exposes write-protection or one-time-programmable memory at the sector level and enforces this protection in hardware rather than purely in software. A platform that only offers software-controlled write protection would allow a sufficiently privileged adversary to disable that protection before overwriting the protected region, defeating the purpose entirely.

5.2.2. Flash and RAM capacity

POC-FUNC-NOMINAL and POC-MEM-LAYOUT together require the flash to accommodate at least the following independent regions: the BSW itself, a primary ASW image (the *MAIN* partition, holding the currently active image) and sufficient additional storage to receive an updated image (the *UPDATE* partition). Each image region must be large enough to hold a complete signed image including its header. The RAM must be large enough to hold the ASW image in working memory during the copy and execution steps of the Nominal Sequence. The exact partitioning strategy — including whether image exchange is performed by physical copying, logical slot designation, or a combination of both — is an implementation choice deferred to Chapter 6.

5.2.3. Protected non-volatile storage for security-critical state

SEC.06 requires a monotonic counter that persists across resets and can only be incremented, never decremented, by software. Dedicated hardware OTP cells might seem a natural fit, but OTP memory is one-time writable by definition: it cannot be updated repeatedly as the counter advances across firmware versions during the mission, making it unsuitable for this purpose. The practical solution is a write-protected flash sector managed exclusively by the BSW. The BSW temporarily lifts write protection only when an authenticated update warrants a counter increment, then immediately re-applies it before transferring control. Crucially, the protection must be enforced at the hardware level so that a compromised ASW cannot circumvent the mechanism by disabling it in software.

5.2.4. Communication channel with ground and the ASW

POC-FUNC-STANDBY requires a communication interface through which ground can issue load, dump and check commands during the Standby mode. The same interface is used to receive the telecommand that determines the mode entered after reset. Additionally, SEC.06 and POC-FUNC-NOMINAL require a shared area of unprotected non-volatile storage through which the ASW can signal its operational status back to the BSW across a system reset. This region, referred to as the COMM partition throughout this work, must be readable and writable by both the BSW and the ASW, but must not be relied upon for security-critical decisions without independent verification.

5.2.5. Cryptographic acceleration - value and trade-offs

POC-PERF-WCET requires that the worst-case boot time be measured and evaluated against mission reconfiguration budgets. Digital signature verification is a significant contributor to boot latency, although the computational cost depends on the chosen signature scheme. Hardware cryptographic acceleration can reduce this latency when the underlying cryptographic primitives are supported by dedicated hardware. Since such accelerators typically target only a subset of primitives, some signature schemes benefit more than others, while unsupported schemes continue to rely on software execution. The resulting trade-off between hardware-accelerated and software-only cryptographic implementations is evaluated empirically in the performance benchmarks of Chapter 7.

5.3. Space-grade hardware overview

The hardware constraints identified in Section 5.2 narrow the candidate platform space considerably, but they do not determine a specific choice. A secondary concern is representativeness: a PoC validated on hardware with no connection to real spacecraft would carry less weight as evidence. This section surveys the processor architectures that have actually been used in space-grade onboard com-

puters, both to confirm the relevance of the selected PoC platform and to identify the natural migration path from the PoC to a flight-ready implementation.

5.3.1. LEON/SPARC - ESA heritage processors

ESA has been developing its own processor line since the 1990s. The LEON family is based on the SPARC V8 architecture and has been the dominant choice for ESA missions requiring radiation-hardened processing. Starting from LEON2-FT, the processors include single-event upset fault tolerance. The current generation, manufactured by Frontgrade Gaisler, includes the GR712RC (dual-core LEON3FT), the GR740 (quad-core LEON4FT) and the upcoming GR765 [22]. These processors are the reference platform for ESA's SAVOIR framework and the target environment for production BSW implementations such as GRBOOT [23].

Despite their heritage and relevance, LEON-based platforms present a significant obstacle for this work. The associated toolchains, BSPs and reference BSW implementations are proprietary and not publicly available. Developing an open-source PoC bootloader from scratch on a LEON platform without access to this material was deemed infeasible. For this reason, LEON-based hardware is considered the intended long-term validation target but is excluded from the PoC platform selection.

5.3.2. ARM Cortex-M in space - adoption and rationale

Alongside the traditional LEON ecosystem, ARM Cortex-M processors have seen growing adoption in space applications, particularly in CubeSats and small satellite missions where cost and development speed are prioritised. Several studies and mission reports confirm the use of STM32-based microcontrollers in space flight software [55, 17, 44, 16], establishing the architecture as a possible choice for onboard computers in space.

Several manufacturers produce ARM Cortex-M devices relevant to this work. Microchip offers the SAMRH71, a radiation-hardened Cortex-M7 processor, alongside the commercial SAMV71 sharing the same core [32]. Vorago offers the VA41630, a radiation-tolerant Cortex-M4 targeting LEO missions, with a next-generation dual-core Cortex-M55 device (the VA5 family) announced in both rad-tolerant and rad-hardened variants [53]. Frontgrade Gaisler, primarily known for the LEON/SPARC line, also produces the UT32M0R500, a radiation-hardened Cortex-M0+ microcontroller [24]. STM32 microcontrollers, while not space-qualified themselves, have been used directly in space flight software [55, 17, 44, 16] and form the commercial baseline.

The practical significance of this lineup is that the radiation-hardened devices from Microchip, Vorago and Frontgrade share the same ARM Cortex-M instruction set architecture as the STM32 family. A BSW implementation developed and validated on a commercial STM32 is therefore portable at the architectural level to the SAMRH71 or VA41630, with differences limited to peripheral register layouts rather than boot logic. The STM32 family is therefore both a flight-proven platform in its own right and a natural proxy for radiation-protected hardware.

5.4. Platform selection for the PoC

With the hardware requirements from Section 5.2 and the space-grade processor landscape from Section 5.3 established, the platform selection becomes a matter of finding the candidate that satisfies all hard requirements while maximising accessibility and representativeness. The underlying principle is to use the most widely available hardware within the ARM Cortex-M ecosystem, avoiding cutting-edge peripherals or non-standard toolchains that would complicate reproduction and may not yet have a track record in space. This directly addresses one of the thesis research sub-questions: what is the minimal level of hardware support necessary for implementing Secure Boot in space-grade systems? If the required primitives are already present in broadly accessible off-the-shelf processors, the path to radiation-hardened variants becomes a porting exercise rather than a fundamental redesign.

5.4.1. Candidate board comparison

Table 5.2 presents the hardware platforms considered for this PoC. The evaluation applied the hardware requirements established in Section 5.2 as the primary filter, but accessibility was also an important consideration. Availability, cost, community support and ease-of-use were also factored in.

Raspberry Pi 5: Included as a common, accessible and widely available candidate, but the BCM2712 processor lacks the publicly available low-level documentation necessary to implement a custom first-stage bootloader, making it incompatible with the SAVOIR requirement that the BSW be the first code to execute after reset.

Raspberry Pi Pico 2: Similarly accessible and low-cost, with the additional benefit of extensive low-level documentation, but the RP2350's internal boot ROM executes unconditionally before any user code, preventing a custom first-stage bootloader from being implemented in compliance with SAVOIR.

GR712RC and PEB1: Relevant as the reference LEON platform, but excluded for the reasons described in Subsection 5.3.1 — limited documentation availability and acquisition difficulties make it unsuitable for an open-source PoC.

SAMV71/SAMRH71: Both boards are strong candidates on technical grounds. They offer flash boot capability, a hardware TRNG, SHA acceleration and (on the SAMV71) AES support. The SAMRH71 is particularly notable as the main radiation-hardened Cortex-M candidate. However, their higher unit cost, more limited availability and smaller open-source community compared to the STM32 ecosystem make them less practical for this PoC. Their architectural proximity to the STM32F4 means the trade-off in representativeness is minimal.

STM32F4 platforms: The STM32F4 family combines the accessibility of the Raspberry Pi boards with the architectural relevance of the other platforms. It is among the most widely used ARM Cortex-M4 families in embedded development, with extensive open-source tooling, broad community documentation and confirmed use in space flight software [55, 17, 44, 16]. Two candidates from this family were considered: the NUCLEO-F401RE and the NUCLEO-F439ZI. Both share the same Cortex-M4 core and boot architecture, but differ significantly in their cryptographic hardware support.

Table 5.2: Microcontroller Board Comparison for Embedded System Selection

Board	HW	Core	Boot	RAM	Flash	Secure Boot HW
Raspberry Pi 5	BCM2712	ARM Cortex A76	N/A	16GB	N/A	-
Raspberry Pi Pico 2	RP2350	ARM Cortex M33	ROM	520kB	4MB	MPU, Boot RAM, TrustZone
SAM V71 XPLAINED	SAMV70/71	ARM Cortex M7	ROM, Flash	384kB	2MB	MPU, TRNG, AES, SHA
SAMRH71F20-EK	SAMRH71	ARM Cortex M7	Flash, Ext PROM	284kB + 32MB	128kB	MPU, TRNG, SHA
NUCLEO-F401RE	STM32F401	ARM Cortex M4	Flash, ROM, SRAM	96kB	512kB	MPU
NUCLEO-F439ZI	STM32F439	ARM Cortex M4	Flash, ROM, SRAM	256kB	2MB	MPU, AES, DES, SHA, HMAC, TRNG
GR712RC	LEON3FT	SPARC V8	ROM	N/A	N/A	-
PEB1	VA41630	ARM Cortex M4	N/A	64 + 256kB	256kB	-
UT32M0R500-EVB	UT32 M0R500	ARM Cortex M0+	ROM/Flash	96kB	64MB	MPU

5.4.2. Selection rationale - NUCLEO-F439ZI

The NUCLEO-F439ZI was selected as the PoC platform on the basis of its STM32F439ZI microcontroller, which satisfies all hardware requirements derived in Section 5.2 while offering the accessibility

and tooling that makes a single-developer research prototype practical.

Memory capacity and layout. The STM32F439ZI provides 2 MB of dual-bank Flash and 256 kB of SRAM, which is sufficient to host all required partitions simultaneously. The Flash is organised into two banks of 1 MB each. Within each bank, the first four sectors are 16 kB, the fifth is 64 kB and the remaining seven are 128 kB. This granular layout maps cleanly onto the partitioning required by POC-MEM-LAYOUT. The MAIN and UPDATE image regions can each be allocated to one or more 128 kB sectors, providing ample space for complete signed images. A small sector is reserved for the rollback counter. The BSW and all security-critical data structures remain in clearly separated, independently protectable regions. The 256 kB SRAM (comprising a 64 kB CCM bank and three further banks of 112 kB, 16 kB and 64 kB) must hold both the BSW itself and the ASW image simultaneously during the Nominal Sequence copy step, as required by POC-FUNC-NOMINAL. The maximum supported ASW image size is therefore 256 kB minus the RAM consumed by the BSW stack, heap and static data; the exact BSW RAM footprint is measured and reported in Chapter 7.

Hardware Root of Trust via option bytes. The STM32F439ZI Flash controller exposes a dedicated set of *option bytes* that configure sector-level write protection, readout protection and debug access. These settings are interpreted and enforced by the Flash controller at boot time, entirely in hardware and cannot be circumvented by software executing at any privilege level. The `WRP` bits allow individual sectors to be marked write-protected, causing any erase or program attempt targeting those sectors to generate a protection error. This mechanism provides the hardware-enforced immutable storage required by SEC.01 and SEC.02: the BSW partition and the public key are placed in write-protected sectors, simulating the ROM that SAVOIR requires, while the BSW itself can verify and restore the protection configuration before proceeding, as it is the first piece of software that is executed after reset (POC-FUNC-INIT).

The same mechanism satisfies SEC.06: the monotonic counter is stored in a dedicated write-protected sector managed exclusively by the BSW. Because write protection is enforced in hardware, the ASW cannot decrement or overwrite the counter; any such attempt is blocked by the Flash controller. If the ASW attempted to lift the write-protection it would need to trigger a system reset, at which point the BSW would re-verify and restore the protection configuration before proceeding.

Cryptographic hardware acceleration. The STM32F439ZI integrates a hardware cryptographic accelerator providing AES, DES/3DES, SHA-1, SHA-224 and SHA-256 in hardware, alongside a hardware True Random Number Generator. The SHA-256 accelerator is directly relevant to the PoC: classical signature schemes such as ECDSA use SHA-256 as their internal hash function, and offloading this operation to hardware measurably reduces verification latency for those schemes. The dedicated hardware CRC unit additionally accelerates the Ethernet CRC calculation used in ASW integrity checks as required by SAVOIR. The relative contribution of hardware acceleration to overall boot latency is quantified empirically in Chapter 7.

Communication interfaces. The device provides four USART/UART peripherals (up to 11.25 MBit/s), two CAN 2.0B controllers, a 10/100 Ethernet MAC with dedicated DMA and USB OTG. This breadth of interfaces satisfies POC-FUNC-STANDBY without constraining the communication channel to a single protocol.

Development environment and tooling. The NUCLEO-F439ZI board integrates an ST-LINK/V2-1 debug probe, eliminating the need for external debug hardware and supporting single-step debugging directly in STM32CubeIDE. The STM32 ecosystem provides ST's HAL and LL driver libraries, STM32CubeMX for peripheral initialisation code generation, and STM32CubeProgrammer for Flash programming and option byte configuration. The combination of comprehensive reference documentation, a large open-source community, confirmed use in space flight software [55, 17, 44, 16], and an integrated debug probe that enables rapid iteration and single-step fault diagnosis makes the STM32F4 family the most practical platform for this PoC.

6

Implementation

This chapter describes the implementation of the secure boot PoC¹ on the STM32F439ZI, mapping the requirements established in Chapter 5 onto concrete design decisions. The chapter is structured around five architectural concerns: memory layout and write-protection strategy (Section 6.2), communication protocol (Section 6.3), the boot state machine and its operating modes (Section 6.4), rollback protection (Section 6.5), and the cryptographic verification interface (Section 6.6).

The implementation prioritises the security requirements from Appendix A. Several ECSS process standards [8, 10, 11] were not followed, as they govern large-team engineering processes out of scope for a single-developer research prototype. However, the Packet Utilisation Standard (PUS) defined in ECSS-E-ST-70-41C [9] was used as the structural basis for the communication protocol, maintaining consistency with the space domain. The focus is on satisfying the PoC requirements established in Chapter 5 (Table 5.1) and the security requirements in Appendix A; safety mechanisms that do not contribute to the security properties under evaluation — such as watchdog timers and redundancy — were not developed as part of this PoC.

6.1. Secure by design

From the outset, the implementation was developed with awareness of the most common vulnerabilities associated with the C programming language. These include, but are not limited to, stack overflows, integer overflows and off-by-one errors. To mitigate these risks, the design avoids dynamically allocated memory wherever possible and validates the length of all received inputs before processing. Function return values are checked throughout, and security-critical logic — option byte checks, signature verification and version control — is placed such that a failure in any step causes the system to abort rather than proceed in a partially trusted state.

6.2. Memory layout

6.2.1. Flash partition structure

The flash memory of the STM32F439ZI is divided into independently erasable sectors. Placing each logical partition in its own sector ensures that erasing or reprogramming one region never corrupts another. The resulting layout is shown in Figure 6.1 and Table 6.1.

The BSW occupies the first 128 kB of flash, which is also the device's default boot address. Flash is then divided into two equal ASW image slots, `SLOT_A` and `SLOT_B`. At any given time one slot is the `MAIN` partition (the currently active image) and the other is the `UPDATE` partition (the candidate image awaiting validation or a backup image used during rollback). Which slot plays which role is determined at boot time by a primary slot index persisted in `PROTECTED_STATE`. The `SWAP` partition supports physical image exchange when needed. The two supported update mechanisms and when each is used are described in Section 6.4. These three regions were set to 128 kB each, matching one flash sector. The

¹The source code is available at <https://github.com/matyaskollert/ESA-SAVOIR-Secure-Boot>.

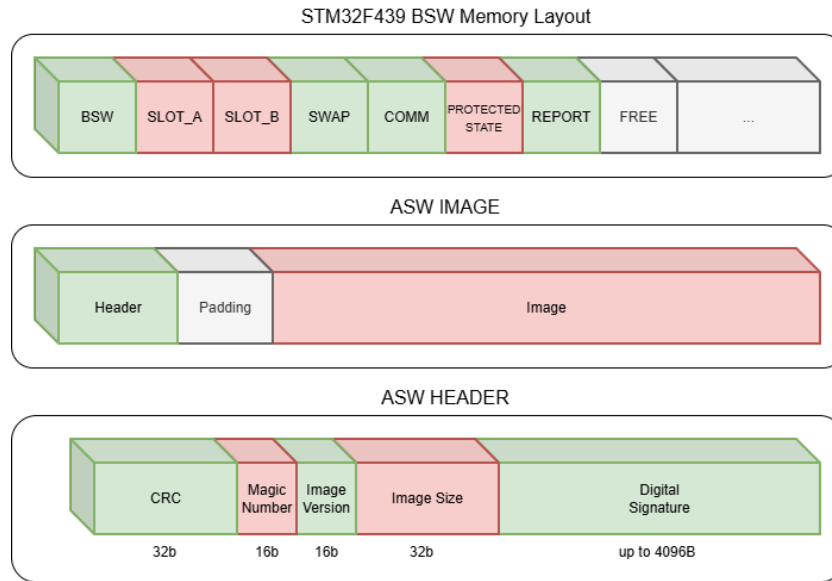


Figure 6.1: Memory layout for the PoC

128 kB ceiling is set by two constraints: a single sector is the minimum independently erasable unit, which is essential for the update and swap operations, and the available SRAM must accommodate the full image during the nominal sequence copy step. `PROTECTED_STATE` stores both the monotonic rollback counter and the primary slot index. The `COMM` partition is a shared, unprotected region used to store the operational status of the BSW across a system reset. Lastly, the `REPORT` partition is reserved for a circular buffer of up to five structured boot reports. Each report is a fixed-size *bsw_report_t* record (20 bytes) written at the conclusion of every significant operation (nominal boot, firmware update, or image swap). The record captures the operation type, outcome code, primary slot identifier at the time of the report, rollback counter value, primary and secondary image versions, and a bitmask of which steps within the operation completed successfully before any failure.

Partition	Sector	Size	WP in nominal mode
BSW	0-4	128 kB	Yes (pre-launch)
SLOT_A	5	128 kB	Yes (when MAIN)
SLOT_B	6	128 kB	Yes (when MAIN)
SWAP	7	128 kB	No
COMM	8	128 kB	No
PROTECTED_STATE	9	128 kB	Yes
REPORT	10	128 kB	No

Table 6.1: STM32F439 PoC Memory Layout

In nominal operation, the BSW, the MAIN slot (whichever of `SLOT_A` or `SLOT_B` is currently primary) and `PROTECTED_STATE` are hardware write-protected via the Flash controller's option bytes, satisfying **SEC.01** and **SEC.06**. The `UPDATE` slot and the `COMM` partition intentionally remain writable by the ASW, as the ASW must be able to receive new images and report its operational status.

The BSW stack and heap are placed in the dedicated core-coupled memory (CCMRAM), which cannot be accessed by the DMA controller, providing isolation between the BSW stack and the DMA receive buffer used for UART communication.

6.2.2. ASW image format

Each ASW image stored in a flash partition consists of a fixed-size header followed by the binary image, as shown in Figure 6.1. The header contains a CRC checksum, a magic number for quick validity detection, a version identifier, the image size and a 4 096-byte signature field. The header and

binary are separated by padding that aligns the image entry point to a fixed offset (5 120 bytes from the start of the partition), simplifying the arithmetic required to locate the image vector table at runtime.

The CRC covers all header bytes following the CRC field itself, plus the full binary image. The digital signature covers the same region, with the signature field zeroed before verifying, so that the signed byte sequence is deterministic. This approach follows the manifest concept defined in the SUIT firmware update architecture [36]: the image header acts as the manifest, carrying the information elements specified in the SUIT information model [35] — version identifier, image size, payload digest and digital signature — in a simplified binary encoding suited to the resource constraints of the PoC platform. The 4 096-byte signature field is sized to accommodate the largest scheme in hybrid mode (ML-DSA-65 at 3 309 bytes plus an ECDSA-P256 signature); a production deployment would fix this to the scheme-specific size determined at mission configuration time.

6.2.3. Option byte configuration

The STM32F439ZI Flash controller enforces sector-level write protection through option bytes that are latched by the hardware at boot time and cannot be bypassed by software executing at any privilege level. In nominal mode, the BSW, the MAIN slot and the PROTECTED_STATE sector are write-protected. As changing option bytes requires a system reset to take effect, any operation that modifies protection state — such as preparing for a swap — immediately triggers a reset and the BSW re-verifies protection on every startup before proceeding to the boot sequence. This prevents an adversary who manages to disable protection in one boot cycle from exploiting that window in the same cycle.

6.3. Communication

While real systems use two redundant processor modules, as described in Subsection 2.4.3, the communication path is simplified for the PoC. In a production system, the active PM running the ASW would receive commands from ground and forward them transparently to the inactive PM running the BSW; since this forwarding layer contributes no logic relevant to the security properties being evaluated, it was omitted without affecting the validity of the PoC.

For the PoC, ground communicates with the board (inactive PM) directly over UART. A Python application running on a desktop PC acts as the ground station, sending commands, uploading images and inspecting the BSW state. All communication between the BSW, ASW and ground uses a common packet protocol implemented in the Board Support Package.

6.3.1. Board Support Package

A shared Board Support Package (BSP) provides three modules used by both the BSW and the ASW.

The **flash module** defines the complete partition memory map and exposes read, write and erase operations, as well as functions to read and write the bootloader status in the COMM partition. It also implements functions that allow any software to identify which slot is the MAIN and which is the UPDATE partition. A digital signature verification interface is exposed through a simple API, allowing any subsequent software to verify further components using the same root of trust. This is particularly relevant on platforms where the first-stage BSW resides in a small or slow external PROM: a minimal stage-one loader in PROM can verify a larger stage-two BSW in write-protected flash and then defer all further verification to the shared BSP interface, extending the chain of trust without duplicating cryptographic logic across stages.

The **packet module** implements a simplified ECSS PUS[9]-inspired protocol. Each packet consists of a 7-byte primary header — carrying a service type, sequence counter, data-field length and an XOR header checksum — followed by a variable-length data field. The header checksum provides lightweight framing integrity detection without significant overhead. Defined service types cover firmware upload (start, data chunks, end), (negative) acknowledgement and debug logging. All `printf()` output is redirected through the debug-packet function so the ground station receives a structured log stream rather than raw ASCII, avoiding ambiguity when binary data is simultaneously in transit on the same UART line.

The **input module** handles UART communication using DMA circular buffering with idle-line detection. This allows the BSW to receive image transfers of arbitrary length without polling and without risking

buffer overruns. As a result, any software can run at higher clock speeds.

Sharing the BSP between the BSW and ASW ensures that both components agree on the partition layout and use the same communication framing, which simplifies integration and testing.

6.4. Boot Process

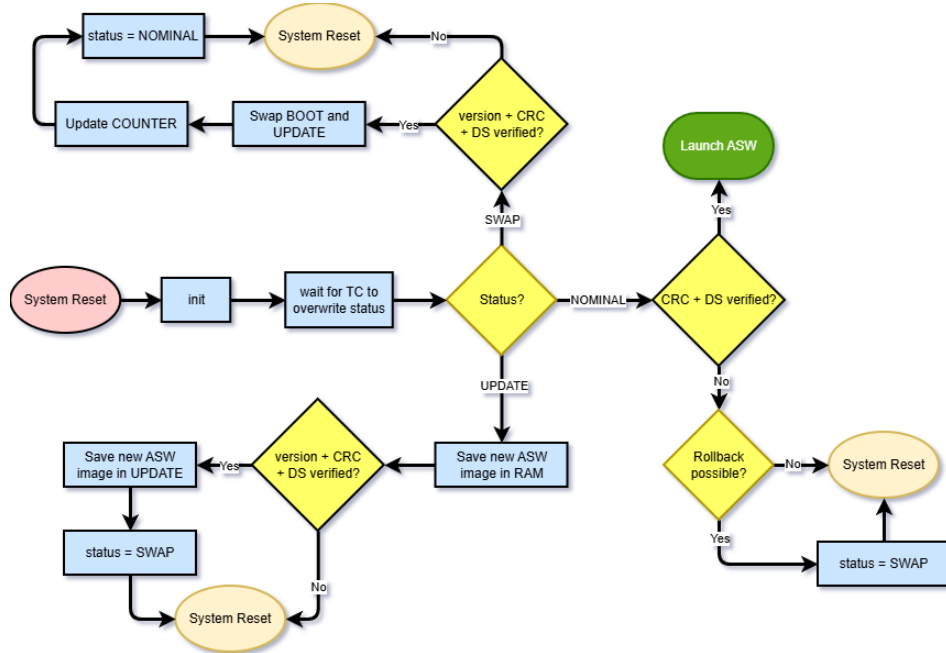


Figure 6.2: Flowchart describing the boot process

On every system reset, the hardware transfers execution directly to the BSW. After peripheral initialisation, the BSW opens a configurable reception window during which ground may send a command to select a specific mode. If no command arrives, the stored COMM status determines the mode entered. If a skip command is received, the stored status is used immediately without waiting for the full window to expire. A simplified flow diagram can be seen in Figure 6.2, while the overall detailed flow is captured in Figures 6.3-6.8.

At the start of each operation — nominal boot, firmware update or image swap — a new report record is created in RAM and its step-flag bitmask is updated incrementally as each phase completes. Writing flags to RAM rather than directly to flash avoids repeated small flash writes and reduces sector wear. At the end of the operation the finalised record is persisted to flash. Because flash must be erased at full sector granularity before new data can be written, all five existing report slots are first read into RAM, the REPORT sector is erased, and then the full buffer (including the new entry) is written back in a single operation. New reports are placed in the first empty slot if one exists. Once all five slots are occupied, the oldest entry is overwritten, implementing a circular buffer.

6.4.1. State management

The BSW's persistent state is a single value written to the COMM partition. The persistent states — nominal, standby, swap, boot_attempted, update, rollback, and report — survive a system reset. Because the COMM partition is writeable by the ASW and therefore untrusted, the BSW never performs any image operations without first verifying their digital signatures. This ensures that the MAIN slot and PROTECTED_STATE cannot be overwritten without a valid image in the UPDATE slot.

6.4.2. Nominal mode flow

The nominal boot sequence enforces the following steps in order, as shown in Figure 6.4.

1. **Protection check.** The BSW verifies that the MAIN slot and PROTECTED_STATE are write-protected.

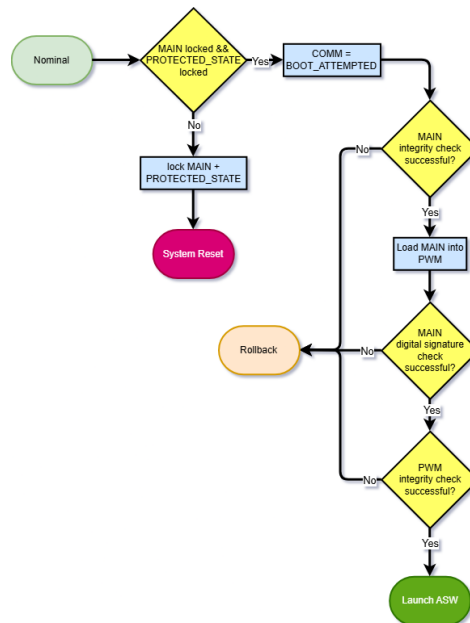


Figure 6.4: Flowchart describing the nominal process

size, a stream of data chunks and an end packet — and written to the UPDATE partition. A version and magic-number check on the first received chunk provides early rejection of obviously invalid images before the transfer completes. After all data has been received, a CRC check and digital signature verification are performed to prevent unnecessarily storing corrupt images. Once the data is written, the BSW configures for a swap and resets. Any error during the update process returns control to standby mode.

An image swap first validates the CRC, digital signature and version of the UPDATE slot before modifying the MAIN slot. Only after all three checks pass does the BSW execute the image exchange. Two mechanisms are supported and the choice between them reflects a trade-off relevant to space-grade hardware.

The **physical swap** performs a three-way copy (MAIN → SWAP, UPDATE → MAIN, SWAP → UPDATE), after which SLOT_A always holds the active image and SLOT_B always holds the previous image. This simplifies the nominal boot path: the BSW always loads from the same fixed flash address without consulting any index. The downside is that every update incurs three full sector erase–write cycles across SLOT_A, SLOT_B and SWAP. Radiation-tolerant flash memories used in space applications typically have a limited erase–write cycle budget, so this penalty is non-trivial over a long mission.

The **logical swap** instead updates only the primary slot index stored in PROTECTED_STATE, making the UPDATE slot the new MAIN without copying any data. Because PROTECTED_STATE is written on every swap regardless (to update the rollback counter), storing the index there adds no additional erase cycle. The three image sectors are left intact, extending their usable lifetime. The cost is a slightly more complex boot path: the BSW must read the index from PROTECTED_STATE before it knows which address to load from.

In both cases the rollback counter is updated, write-protection is restored to reflect the new MAIN slot and the system resets. If any of the pre-swap checks fail, the BSW configures the system for nominal boot mode and restarts into standby mode.

6.4.4. Integrity and signature verification sequence

The separation of CRC and signature verification, and their ordering relative to the flash-to-RAM copy, is intentional as it follows the SAVOIR requirements. The CRC is evaluated in flash first because it is cheap and detects silent flash corruption or uninitialised partitions before committing to a memory copy. The signature is evaluated only after the copy into RAM, as the digital signature field has to be

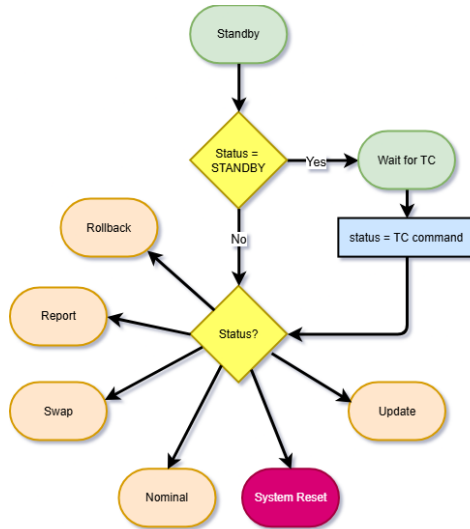


Figure 6.5: Flowchart describing the standby process

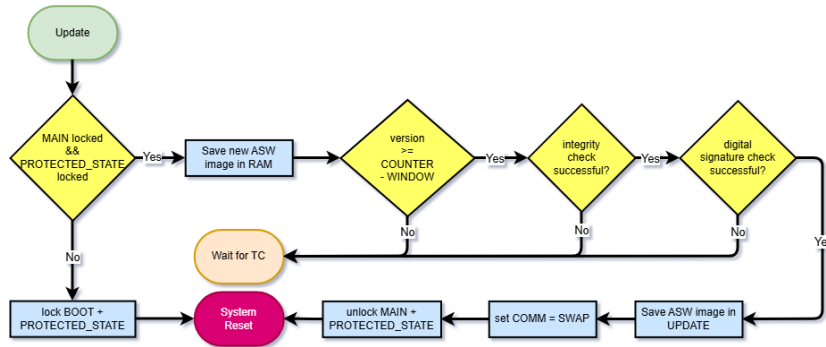


Figure 6.6: Flowchart describing the update process

modified, which would be impossible purely in flash. Zeroing the signature field before verifying and restoring it afterwards ensures that the byte sequence fed to the verification function is the same as what was used for signing the image in the first place. A secondary CRC check is performed before execution, to ensure that the transfer did not cause any memory corruption.

6.5. Rollback protection implementation

To satisfy **SEC.06**, three candidate designs for rollback protection were evaluated. All designs share one hard constraint: at least one partition must remain writable by the ASW to support autonomous in-orbit updates.

6.5.1. Option A - version-based window, no counter

Under this scheme, there is no independent counter, but the rollback floor is derived directly from the version of the image currently in the MAIN partition, i.e. $v_{\min} = v_{\text{MAIN}} - X$. This design is vulnerable to an iterative downgrade attack. Consider a system running version N with window $X = 1$. The attacker uploads version $N - 1$ to the UPDATE slot and triggers a rollback. Since $N - 1 \geq N - 1$, the rollback is within the permitted window and is accepted. After the swap, MAIN now contains version $N - 1$. Because the floor is re-derived from the new MAIN version, it has also decreased to $N - 2$. The attacker can therefore repeat the attack indefinitely, reducing the running version by one step per cycle until an arbitrarily old and vulnerable image is reached. Option A was rejected for this reason.

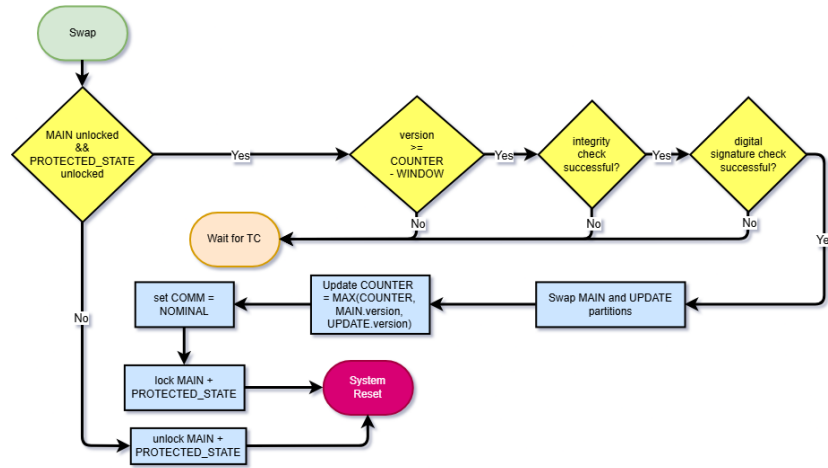


Figure 6.7: Flowchart describing the swap process

6.5.2. Option B - two partitions with monotonic counter

Option B addresses the vulnerability in Option A by storing the rollback floor in a separate, hardware-protected monotonic counter rather than deriving it from the mutable MAIN partition. The counter records the highest version that has ever been successfully executed and can only be incremented, never decremented, because it resides in a write-protected sector that only the BSW may modify. The rollback floor is $v_{\min} = N - X$, where N is the counter value.

Under the same attack scenario, an attacker who triggers one permitted downgrade from version N to $N - 1$ does not lower the counter, because the counter was set to N when version N was first executed. The floor therefore remains at $N - 1$, and the next attempted downgrade to $N - 2$ is rejected. The attack is thus bounded to a single step regardless of how many times the attacker controls the update channel. To perform a second downgrade, the attacker would first have to install and successfully execute a version higher than N , which requires a valid signature from the private key.

6.5.3. Option C - three partitions, no counter

A three-partition design can prevent iterative downgrade without a dedicated counter by separating the rollback and update code paths such that neither path can reach an arbitrarily old version through a sequence of individually valid operations. This option avoids the counter dependency at the cost of an additional flash partition and somewhat more complex swap logic.

In this scenario, both SLOT_A and SLOT_B are write-protected and a new SLOT_C image partition is used to receive updates instead. To transfer an image to one of the main slots, its version and digital signature need to be verified, ensuring that an invalid image or an image version outside of the rollback window cannot be placed in SLOT_A nor SLOT_B. As the rollback operation swaps these protected partitions, the attack vector from Option A is no longer applicable even without a dedicated rollback counter.

6.5.4. Selected approach and rationale

While both Options A and B are a valid implementation, Option B was selected as it provides strong, formally bounded rollback guarantees and is well-matched to the available hardware: the option-byte write protection of the STM32F439ZI makes implementing a dedicated counter sector straightforward.

The rollback window X is set to 1, permitting recovery to the immediately preceding version while bounding any downgrade to a single step. This value is a compile-time constant defined in the BSW configuration. Missions requiring a different window size can adjust it at build time without modifying any other boot logic. The rollback floor is defined as:

$$v_{\min} = \max(0, N - X) \quad (6.1)$$

where N is the monotonic counter value. After any successful swap, the counter is updated to the maximum of N , v_{MAIN} , and v_{UPDATE} , ensuring strict monotonicity. Before initiating a rollback, the BSW

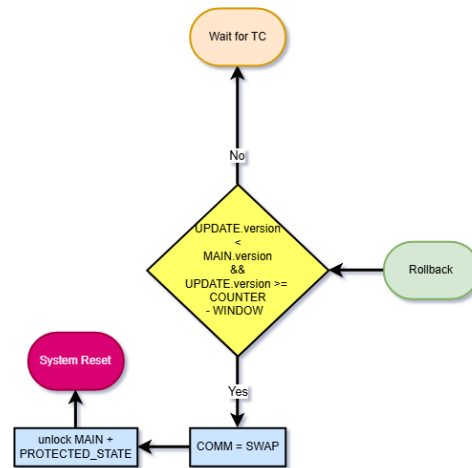


Figure 6.8: Flowchart describing the rollback process

verifies two conditions: that the UPDATE version is strictly lower than the MAIN version (confirming the operation is a genuine downgrade) and that it satisfies Equation 6.1. Both conditions must hold simultaneously — failing either causes the BSW to enter the standby command loop and await ground intervention.

6.6. Cryptographic interface

The cryptographic choices follow the algorithm recommendations discussed in Subsection 2.4.6. ECDSA-P256 and ML-DSA-65 are integrated into the PoC as the primary classical and post-quantum signature schemes, respectively. ECDSA was selected as the representative elliptic-curve-based signature scheme, while ML-DSA was selected as the representative lattice-based post-quantum signature scheme.

RSA-4096 and LMS are additionally included in the benchmarking campaign. RSA serves as a representative of traditional factorisation-based signatures, while LMS was selected as the representative hash-based post-quantum signature scheme. Compared to other hash-based schemes such as XMSS and SLH-DSA, LMS offers comparable security with favourable performance characteristics, making it a suitable candidate for resource-constrained environments [56]. Together, the selected algorithms represent the major families of digital signature schemes: factorisation-based (RSA), elliptic-curve-based (ECDSA), lattice-based (ML-DSA), and hash-based (LMS).

Key lengths are set in accordance with the recommendations discussed previously. The 4,096-byte signature field in the image header is sized to accommodate the largest expected signature (ML-DSA-65 at 3,309 bytes) alongside an ECDSA-P256 signature in hybrid mode, ensuring that the header format does not need to change when switching algorithm configurations.

6.6.1. Algorithm-agnostic verification API

The cryptographic module exposes a single `verifySignature()` function that accepts the signed byte buffer, its length and a pointer to the signature buffer. The calling code in the boot sequence is identical regardless of which algorithm is active. All algorithm selection is resolved at compile time through preprocessor defines.

The public keys are compiled as constant data directly into the BSW binary, placing them in the same write-protected flash sectors as the BSW code itself and satisfying **SEC.02** without any additional key storage mechanism. An alternative is to store only a hash of the public key in immutable memory and include the full key with each image upload; the BSW then hashes the received key and compares it against the stored hash before use, reducing the immutable storage requirement to 32–48 bytes regardless of algorithm. However, the full key must then be transmitted with every update and stored alongside each image in regular flash. For the PoC, which uses a single fixed key, storing the full key directly in write-protected flash is the simpler and equally secure choice.

All cryptographic primitives are provided by the wolfCrypt library rather than custom implementations, following the principle that cryptographic code should not be written from scratch without extensive validation. The `verifySignature()` function is also exposed through the BSP, so any subsequent software component — a stage-two bootloader or a dynamically loaded module — can verify further components using the same root of trust without duplicating cryptographic logic.

6.6.2. Classical algorithms - ECDSA

The classical implementation uses ECDSA-P256 with the wolfCrypt library, in compliance with the recommendations discussed in Subsection 2.4.6. The public key is stored in standard DER Subject-PublicKeyInfo encoding. SHA-256 is computed over the signed buffer region and the resulting digest is passed to the signature verification function. The DER encoding carries the signature length explicitly, so signatures of varying length can be handled correctly without knowing the signature in advance.

6.6.3. Post-quantum algorithms - ML-DSA

The post-quantum implementation uses ML-DSA-65 (NIST FIPS 204 [46], security level 3), also via wolfCrypt. Unlike ECDSA, ML-DSA operates directly on the full message and performs its own internal hashing, so no pre-computation of a digest is required.

6.6.4. Hybrid algorithms - ECDSA + ML-DSA

In hybrid mode, both algorithms must succeed for the image to be accepted, as recommended during the post-quantum transition period [28]. The 4 096-byte signature field is partitioned so that the ECDSA signature occupies the first 72 bytes (the maximum DER-encoded length for P-256) and the ML-DSA signature occupies the following 3 309 bytes, for a combined total of 3 381 bytes. ECDSA verification is performed first; if either verification fails, the image is rejected immediately.

The algorithm-agnostic API, compile-time algorithm selection and wolfCrypt-backed primitives together ensure that switching between classical, post-quantum and hybrid deployments requires only a build-flag change, with no modifications to the boot sequence logic or the image format. This directly satisfies **SEC.05** and leaves the door open for adopting future NIST-standardised algorithms as they become available in the library.

7

Evaluation

7.1. Evaluation methodology

The evaluation of the PoC is structured around two complementary concerns: security correctness and resource overhead. For security correctness, the primary method is end-to-end automated testing that drives the BSW through all operationally significant scenarios and verifies the observable outcome against the expected behaviour. For resource overhead, performance benchmarks and static memory analysis provide concrete data for mission designers.

Verification methods mandated by ECSS and SAVOIR — review panels, controlled inspection against qualification documentation, test execution against a qualification model under a traceable test plan — were not applied, as they govern large-team engineering processes that are out of scope for a single-developer research prototype. The approach taken here is therefore intentionally lighter-weight: automated behavioural tests and manual code review serve as the primary evidence of correctness. This is acknowledged as a limitation in Section 7.4.

The security analysis in Section 7.2 verifies the state machine against the flow diagrams and maps each automated test to the corresponding PoC requirement. The performance evaluation in Section 7.3 reports signature verification latency for four schemes under hardware-accelerated and software-only SHA configurations, as well as the static memory footprint of each scheme.

7.2. Security analysis

In order to verify the correctness of the PoC, the flow diagrams from Figures 6.3-6.8 were used as the primary reference. All states and transitions were enumerated, and the expected behaviour of each was documented before any code was written. The BSW was then implemented to follow the diagram as closely as possible.

7.2.1. Coverage of attacker scenarios from threat model

With the flow diagram verified, an end-to-end automated test suite was developed to confirm that the implemented BSW behaves correctly across all operationally significant scenarios. The test suite is implemented in Python using pytest and communicates with the board over two independent channels: UART for ECSS-compatible telecommand and telemetry packets, and SWD via the STM32CubeProgrammer CLI for direct flash and option-byte access. The dual-channel approach is essential: it allows the tests to configure preconditions (flash content, write-protection state, COMM status word) that the BSW itself would never set, and to inspect post-conditions through a trusted path that does not rely on the BSW reporting correctly about itself.

Before each test, the board is brought into a fully known state. The relevant flash sectors are erased or programmed with prepared images, the option bytes are set to the required write-protection configuration, and the COMM status word is written directly to flash. The BSW is then triggered by sending the appropriate commands over UART. After the BSW has responded (ACK, NACK, or timeout), the test

reads the REPORT flash sector, the COMM word, the option-byte state, and other data/configurations from flash directly over SWD to verify the outcome.

The test suite is organised into five modules. `test_boot.py` covers the nominal boot path: it verifies that a valid signed image boots successfully, and that corrupted images (missing header, bad CRC, invalid signature, bad magic) are rejected with the correct outcome codes in the BSW report. It also verifies that write-protection violations are detected before any image check is attempted, and that the BSW automatically re-applies the correct protection state before resetting. `test_update.py` covers the firmware upload path: it verifies the happy path, version-floor rejection (with and without boundary conditions), and all WRP violation scenarios for the upload target slot. `test_swap.py` tests the image swap command in isolation, including correct counter advancement, rejection of corrupt or missing update slots, and all five WRP error cases. `test_lifecycle.py` combines multiple operations into end-to-end upgrade cycles, including two consecutive upgrades ($v1 \rightarrow v2 \rightarrow v3$), rollback within the allowed window, and automatic rollback triggered by simulated boot failures. `test_protocol.py` verifies robustness against malformed or out-of-order ECSS packets, ensuring the BSW never hangs and never silently corrupts the primary image slot.

A key invariant verified across all tests is that the BSW report written to flash after each operation accurately reflects which steps completed and, in failure cases, the exact failure cause.

7.2.2. Requirement verification (per SEC-* requirements)

The SAVOIR-GS-002S specification (listed in full in Appendix A) assigns a formal verification method to each requirement: Review of Design (ROD), Inspection (I), or Test (T). The PoC requirements derived in Chapter 5 carry the same categories implicitly. It is important to note that these verification methods are not followed in the rigorous manner that a space-qualified project would. In a production setting, ROD would involve a formal review panel, inspections would be conducted against controlled design documentation, and tests would be executed against a qualification model under a traceable test plan. In this work, verification is carried out informally through end-to-end automated tests and code review, which is weaker than what the SAVOIR standards would require in practice. Table 7.1 and Table 7.2 summarise how each requirement is addressed in the PoC under this lighter-weight approach.

Req.	Description	Evidence in PoC
<i>Group 1 — Security requirements (SAVOIR-GS-002S)</i>		
SEC.01	Hardware Root of Trust	BSW resides in write-protected sector; option-byte WRP verified at each boot by <code>performSelfTests()</code>
SEC.02	Public Key in Immutable Memory	ECDSA-P256 public key stored in write-protected flash alongside the BSW
SEC.03	ASW Digital Signature Verification	<code>TestBootValidImage</code> , <code>TestBootInvalidSignature</code> , <code>TestSwapBadUpdateSlot</code>
SEC.04	Rejection of Unauthenticated ASW	<code>TestBootInvalidSignature</code> , <code>TestBootNoImage</code> , <code>TestBootBadMagic</code> , <code>TestBootCorruptCRC</code>
SEC.05	Classical + Post-quantum Support	ECDSA-P256 (classical) and ML-DSA available at build time along with a hybrid option
SEC.06	Rollback Protection (version window)	<code>TestUpdateVersionTooLow</code> , <code>TestRollbackPrevention</code> , <code>TestRollbackCounterBoundary</code> , <code>TestSwapRollbackEnforcement</code> , <code>TestRollbackWithWindow</code>
SEC.07	Secure Boot Failure Reporting	Every test in <code>test_boot.py</code> , <code>test_update.py</code> and <code>test_swap.py</code> validates the REPORT flash record; outcome codes, step-flag bitmasks and slot identifiers are all checked

Table 7.1: Requirement coverage summary — security requirements (SAVOIR-GS-002S).

Req.	Description	Evidence in PoC
<i>Group 2 — Functional requirements (SAVOIR-derived)</i>		
POC-FUNC-INIT	Hardware initialisation, self-tests and WRP verification before Nominal Sequence	TestBootUnprotectedMainSector, TestBootUnprotectedBSWStateSector, TestBootBothSectorsUnprotected, TestBootNominalAutoFix; WRP state verified via SWD option-byte read after each reset
POC-FUNC-NOMINAL	Full nominal sequence: image selection, CRC integrity check, signature verification, RAM copy, execution transfer	TestBootValidImage (happy path); TestBootCorruptCRC, TestBootBadMagic, TestBootNoImage (integrity failures); TestBootInvalidSignature (authentication failure)
POC-FUNC-STANDBY	Standby mode: ground-triggered entry, memory inspection, firmware upload via ECSS packet protocol	TestUpdateHappyPath, TestUpdateStateAfterSuccess; upload-to-swap lifecycle verified in TestFullUpdateCycle and TestConsecutiveSwaps
POC-FUNC-REPORT	Boot Report written progressively to protected flash after every significant operation	All tests in test_boot.py, test_update.py and test_swap.py read and validate the REPORT sector after each run; magic, type, outcome, step-flag bitmask, slot and counter fields are all checked
<i>Group 3 — Memory architecture requirements (SAVOIR-derived)</i>		
POC-MEM-LAYOUT	BSW and public key in write-protected Boot Memory; ASW, update, swap, COMM and state regions in separately allocated flash sectors	Verified by inspection of the linker script (.ld) and STM32CubeIDE Build Analyser output; sector assignments confirmed via SWD flash map read
<i>Group 4 — Design and performance constraints</i>		
POC-IMPL-DESIGN	Sequential bare-metal execution; no OS or runtime dependency; minimum resource usage	Verified by code review: no RTOS calls, no dynamic allocation in the boot path, single-threaded control flow throughout
POC-PERF-WCET	Worst-case verification time measured per signature scheme	Benchmark results reported in Table 7.3; four schemes measured at 1 kB and 128 kB image sizes with and without hardware SHA-2 acceleration

Table 7.2: Requirement coverage summary — functional, memory and design requirements (SAVOIR-derived).

7.3. Performance benchmarks

7.3.1. Benchmark setup - hardware, toolchain, methodology

Benchmarks were performed on the same NUCLEO-F439ZI hardware used in the proof of concept. The main clock was configured to 168 MHz to match the deployment setup. For benchmarking purposes, only the relevant code was extracted, namely the image digital signature verification logic, which remained identical to the implementation used in the PoC. Four signature schemes were evaluated: ECDSA, RSA, ML-DSA, and LMS, selected based on the recommendations described in Subsection 2.4.6. Any of these implementations can be integrated at build time. For a hybrid deployment, the total verification time is the sum of the individual scheme verification times, while the working memory requirement corresponds to the larger of the two memory footprints.

These benchmarks are dependent on the cryptography library that is being used. Even within a single library, there are many ways to change how the computations are done based on whether you want to optimise for memory, speed, or other criteria. The specific wolfSSL configuration used for all benchmarks is listed in full in Appendix B. Any changes that were made between runs are clearly stated

below. Unnecessary wolfCrypt features — key generation, signing and key export — were disabled at compile time to reduce the code footprint to the verification path only.

For each signature scheme, three runs were performed and the result reported is the average of the measurements. There was negligible variation between runs when the same data was used.

The relationship between image size and verification time differs across schemes. ECDSA and RSA are hash-then-sign schemes: they first compute a SHA-2 digest of the entire image and then perform the signature operation on that fixed-length hash. For these schemes, only the hashing step scales with image size; the signature operation itself is constant-cost. LMS verification involves three steps: a randomised message hash (`H_msg` applied over the full image, which scales linearly with image size), followed by constant-cost Winternitz one-time signature chain traversal, and a constant-cost Merkle authentication path check. The Winternitz chain operations dominate LMS verification time and are fixed for a given parameter set; the image-dependent `H_msg` step is a comparatively small addition. ML-DSA is fundamentally different: it uses SHAKE-256 (a XOF from the SHA-3 family, as specified in FIPS 204 [46]) internally and absorbs the full message directly into the sponge rather than pre-hashing it, so its cost grows proportionally with message length with no separation between the hashing and signature steps.

For ECDSA, RSA, and LMS, each benchmark was run with two SHA-2 configurations: one using the STM32 hardware hash accelerator (HASH peripheral, referred to as HW SHA) and one using a software SHA-2 implementation (SW SHA). The STM32 HASH peripheral accelerates only SHA-2 (and MD5); it has no support for SHAKE-256. Because ML-DSA uses SHAKE-256 internally rather than SHA-2, the hardware accelerator offers no benefit and ML-DSA was only measured in the software configuration. Image sizes of 1 kB and 128 kB were used, where 128 kB is the maximum ASW image size in the PoC. The 1 kB case approximates a minimal image and exposes the fixed overhead of the signature scheme, while the 128 kB case reveals how much the cost grows with image size.

7.3.2. Signature verification latency

Table 7.3 reports the average signature verification time for each scheme across three identical runs (variation was negligible). All results are specific to this wolfSSL configuration on the STM32F439ZI running at 168 MHz; a different library, a different set of compile-time flags, or a different processor could shift the absolute numbers significantly. What matters for hardware selection is therefore the relative ordering between schemes, which is more stable, provided that the same degree of hardware acceleration is available on the target platform.

With hardware SHA-2 acceleration (HW SHA). When the STM32 HASH peripheral is enabled, RSA is the fastest classical scheme by a considerable margin: RSA-3072 completes in only 38 ms for a 1 kB image and 46 ms for 128 kB, outperforming ECDSA-P256 (177 ms / 184 ms) by a factor of approximately four to five. ECDSA's verification time is dominated by the elliptic-curve scalar multiplication, so the SHA-2 acceleration has almost no effect on it; indeed, the 1 kB and 128 kB times are nearly identical, with the 7 ms difference coming entirely from the image hash. For RSA, in contrast, HW SHA provides a meaningful reduction on large images (e.g., RSA-3072 drops from 101 ms to 46 ms at 128 kB compared to the software path). LMS with HW SHA takes 144 ms for 1 kB and 179 ms for 128 kB; notably, LMS with HW SHA is slower than LMS with SW SHA on small images (144 ms vs 132 ms), suggesting that the per-call overhead of the hardware peripheral is significant relative to the short SHA-2 computations inside the Winternitz chains. ML-DSA is not applicable in this column because it uses SHAKE-256 internally, which is not supported by the STM32 HASH peripheral.

Without hardware SHA-2 acceleration (SW SHA). When all hashing runs in software, ML-DSA becomes the fastest scheme overall. ML-DSA-44 verifies a 1 kB image in just 22 ms and ML-DSA-65 in 32 ms, both well ahead of RSA-3072 (39 ms). However, because ML-DSA's cost scales linearly with message length (it absorbs the full image into a SHAKE-256 sponge), the 128 kB times are considerably higher: 137 ms for ML-DSA-44 and 148 ms for ML-DSA-65. At this image size RSA-3072 (101 ms) and RSA-4096 (129 ms) are actually faster. ECDSA with SW SHA (177 ms / 241 ms) is the slowest classical scheme and degrades more at large images because the software SHA-2 path is slow relative to the hardware path. LMS with SW SHA is 132 ms for 1 kB (faster than HW SHA due to lower per-call overhead) but 220 ms for 128 kB, where the software SHA-2 path for `H_msg` becomes the bottleneck.

For a hybridised deployment (e.g., ECDSA + LMS), the total latency is the sum of the two individual times, since both verifications must pass. On the 128 kB image with HW SHA this gives approximately $184 + 179 = 363$ ms, which is still well within a single-second boot budget.

Algorithm	Time with HW SHA (ms)	Time with SW SHA (ms)
ECDSA-P256	1kB: 177, 128kB: 184	1kB: 177, 128kB: 241
RSA-3072	1kB: 38, 128kB: 46	1kB: 39, 128kB: 101
RSA-4096	1kB: 65, 128kB: 73	1kB: 66, 128kB: 129
ML-DSA-44	N/A	1kB: 22, 128kB: 137
ML-DSA-65	N/A	1kB: 32, 128kB: 148
LMS	1kB: 144, 128kB: 179	1kB: 132, 128kB: 220

Table 7.3: Digital Signature Verification Times

LMS verification is faster without hardware-accelerated SHA-2 on small images, where the per-call overhead of the HASH peripheral outweighs the benefit, but hardware acceleration is advantageous on larger images. LMS is also the only post-quantum scheme that BSI and ANSSI currently recommend for standalone use without hybridisation, which gives it a regulatory advantage that the latency numbers alone do not capture.

7.3.3. Classical vs PQ comparison and boot time impact

Contrary to common expectation, some post-quantum algorithms perform better than their classical counterparts on this platform. As shown in Table 7.3, both LMS and ML-DSA outperform ECDSA. For small images (1 kB) and without hardware SHA acceleration, the stronger ML-DSA-65 also outperforms RSA-3072.

7.3.4. Code size and memory footprint

The static code size can be measured precisely from the build output. Most of the binary resides in flash, with a small initialised-data section placed in RAM. At runtime, the BSW also uses RAM to hold all five `bsw_report_t` records (5×20 bytes = 100 bytes) simultaneously. Because flash must be erased at sector granularity before new data can be written, all existing reports must first be read into RAM, the sector erased, and then all reports (including the new one) written back. Keeping the full circular buffer in RAM throughout the operation ensures that no existing reports are lost during the erase cycle.

The wolfCrypt library can be configured to use no dynamic allocation, which means the heap footprint is minimal and the dominant runtime cost is stack usage. As the ASW takes up at most 128 kB, it leaves the system with half of the available RAM for the bootloader. The numbers below are specific to the wolfSSL configuration used for these benchmarks. Compile-time flags allow trading memory for speed (e.g., precomputed tables) or vice versa, so different configurations could shift the footprint numbers noticeably.

These approximate measurements in Table 7.4 describe how much memory each signature scheme requires with the cryptographic library used. The main components are the public key size (stored in protected flash), the signature size (embedded in the image header), the key structure held in RAM, and the working memory (stack and heap, placed in CCMRAM) used during the verification call.

Static memory details can be obtained from the STM32CubeIDE Build Analyser and Static Stack Analyser tools. The memory structure is defined in the linker script (LD file), which specifies where each section of the binary is placed. This makes it straightforward to read how much flash, RAM and CCMRAM the BSW occupies. Stack and heap usage is not easy to bound statically, so the minimum working memory size for each verification algorithm was determined at runtime by running each scheme with progressively smaller CCMRAM allocations until a memory fault occurred.

Table 7.4 shows the memory cost of each algorithm's verification path. The public key (stored in protected flash) and the signature (embedded in the image header) contribute to the static flash footprint, while the key structure held in RAM and the working memory used during verification define the dynamic cost.

LMS has by far the smallest flash and RAM footprint: 60 bytes for the public key, 128 bytes for the

key structure, and only 3 kB of CCMRAM during verification. This makes it the most attractive option when memory is severely constrained, for example on a dedicated stage-one bootloader residing in a small PROM. ML-DSA requires 11–12 kB of CCMRAM during verification, but its RAM key structure is modest at 512 bytes. ECDSA has a small public key (91 bytes) but a relatively large RAM structure (4 kB). RSA-4096 requires 8 kB of CCMRAM with a 550 byte public key.

For missions with extremely limited RAM, LMS is the only option that fits in 4 kB. All other schemes require at least 5 kB of working memory. If memory is plentiful, any scheme is viable and the choice should be made on the basis of regulatory requirements and expected cryptographic lifetime.

It is also worth noting that the public key need not be stored in its raw form in immutable memory; the trade-off between storing the full key and storing only a hash is discussed in Subsection 6.6.1.

Algorithm	PubKey (FLASH)	RAM	Signature	CCMRAM (stack + heap)
ECDSA-P256	91B	4kB	72B	5kB
RSA-3072	2kB	425B	384B	7kB
RSA-4096	550B	2kB	512B	8kB
ML-DSA-44	1312B	512B	2420B	11kB
ML-DSA-65	1952B	512B	3309B	12kB
LMS	60B	128B	1296B	3kB

Table 7.4: Digital Signature Memory Footprint

7.4. Limitations and threats to validity

Several limitations reduce the strength of the conclusions that can be drawn from the evaluation.

BSW not in ROM. To allow for rapid development iteration, the BSW is stored in regular flash rather than a true boot ROM. This means that an attacker with physical access to the board can erase and replace the BSW without requiring a valid signature, undermining the hardware Root of Trust. In a production system, the bootloader would reside in the device boot ROM or in a write-once PROM that cannot be reprogrammed after manufacture. Additionally, the STM32F439ZI exposes a factory bootloader through its BOOT0 pin; with physical access, this allows the device to be returned to a factory-clean state regardless of the option-byte configuration. Replacing the built-in bootloader with the custom BSW and permanently disabling the BOOT0 path would eliminate this vector in a production deployment.

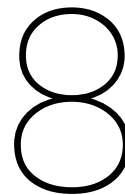
Informal verification. All requirement verification was performed through automated end-to-end tests and code review rather than through the formal review, inspection and test methods defined by ECSS. The automated tests cover the complete set of nominal and failure paths defined in the state machine, but they cannot substitute for the rigour of a formally controlled qualification campaign. Edge cases in the flash programming logic, race conditions during option-byte application, or subtle state corruption in the COMM partition under adversarial timing are examples of failure modes that behavioural tests may not detect.

Single hardware platform. All benchmarks and functional tests were conducted exclusively on the STM32F439ZI running at 168 MHz. Absolute verification times and memory footprints are specific to this processor, its Flash controller, its HASH peripheral and the wolfSSL compile-time configuration used. Results may differ significantly on other microcontrollers with different CPU architectures, cache sizes, or hardware acceleration capabilities. The relative ordering between schemes is more stable, but should still be validated on the target platform before a deployment decision is made.

Subset of the attack surface. The test suite focuses on the BSW state machine and the cryptographic verification path. It does not test the underlying wolfCrypt library for correctness, cover timing side-channels in the signature verification routines, or address fault-injection attacks that could flip a branch outcome or corrupt a register during verification. These are relevant concerns for production deployments on platforms without dedicated fault countermeasures.

7.5. Evaluation summary

The evaluation results taken together provide a consistent picture. The automated test suite demonstrates that the BSW state machine behaves correctly across all nominal and failure scenarios, and that each of the seven SEC requirements is satisfied by the implementation. The requirement traceability tables in Table 7.1 and Table 7.2 show that every requirement is covered by at least one concrete test or inspection artefact. The performance benchmarks show that the security overhead added by digital signature verification is modest on representative COTS hardware: even the slowest scheme (ECDSA-P256 with software SHA at 128 kB) completes in well under one second, and a hybrid ECDSA plus LMS deployment stays below 400 ms with hardware acceleration. The memory footprint analysis confirms that all evaluated schemes fit within the resource envelope of the target platform, with LMS standing out as the best option for extremely constrained deployments. In combination, these results support the thesis claim that Secure Boot is feasible on space-representative COTS hardware without imposing unacceptable reliability or performance penalties.



Related Work

The secure boot implementations described in this chapter have influenced the design decisions made in this work. None of them can be adopted directly for space use: their underlying bootloaders target different processor architectures (x86, ARM Cortex-A), rely on hardware peripherals and memory configurations that differ significantly from radiation-tolerant space-grade platforms, and operate under resource and reliability constraints that are incompatible with those described in Section 2.4. However, the secure boot mechanisms layered on top of those bootloaders — signature verification schemes, chain-of-trust models, rollback protection strategies and update protocols — are architecture-independent concepts that transfer well. Where an implementation is more sophisticated than what is required for the baseline missions targeted by this work, it is noted as a direction for future work in Chapter 9.

8.1. Secure boot in terrestrial systems

8.1.1. Android Verified Boot

Android provides one of the most thoroughly documented open-source examples of a secure bootloader for a constrained, Linux-based system [25]. Its boot flow combines hardware trust establishment, partition integrity verification and A/B slot selection into a single coherent sequence, mirroring the chain-of-trust model discussed in Subsection 2.2.1. Verified Boot uses a hash tree for partitions that are too large to verify entirely before execution, allowing on-the-fly block-level integrity checking. Rollback protection is enforced by binding security keys to the OS version and security patch level, so that downgrading to a vulnerable firmware version does not grant access to keys generated under the newer version [25].

The A/B update mechanism [26] with its automatic slot fallback is a practical realisation of the reliability requirements discussed in Subsection 2.1.2. The underlying bootloader and its TEE dependencies are not applicable to space-grade hardware, but the concepts of hardware-anchored trust, partition-level integrity verification and version-bound rollback protection directly inform the design of this work.

8.1.2. UEFI Secure Boot

On consumer computers, the Unified Extensible Firmware Interface (UEFI) [52] has been the standard firmware interface since 2006. It is responsible for initialising hardware, loading device drivers and providing a uniform interface for subsequent boot stages, after which it hands off to a second-stage bootloader such as GRUB or the Windows Boot Manager via a shim layer.

UEFI Secure Boot establishes a hierarchical key infrastructure: a Platform Key (PK) stored by the OEM in tamper-resistant non-volatile storage acts as the trust anchor, from which Key Exchange Keys (KEK) can be derived to authorise operating system and driver signatures [34]. Keys that are later revoked are placed in a Forbidden Signature Database. Microsoft requires firmware to be signed with at least RSA-2048 and SHA-256 [33], which is below the key-length recommendations for space systems discussed in Subsection 2.4.6.

The hierarchical key model and the use of a signature database are interesting concepts for missions that involve multiple software providers or frequent third-party updates. However, the full UEFI infrastructure is overly complex for the single-vendor, resource-constrained scenarios targeted by this work; a simpler single-key model is preferred here. As discussed in Subsection 9.3.2, a UEFI-like key hierarchy could be explored as future work for more complex missions.

8.1.3. wolfBoot

wolfBoot [57] is an open-source secure boot bootloader for embedded devices developed by WolfSSL. It supports digital signature verification, rollback protection and OTA updates across a wide range of microcontroller targets. Software images follow the RFC 9019 [36] manifest model, with a header containing version metadata used to enforce rollback protection before an update is applied. Updates are managed through a three-partition scheme (BOOT, UPDATE and SWAP) that preserves the previous image as a fallback, consistent with the A/B concepts discussed in Subsection 2.1.2.

The secure boot concepts in wolfBoot — manifest-based image metadata, version-enforced rollback protection and signature verification using a well-audited cryptographic library — are directly relevant to this work. Its broad platform support, however, comes at the cost of considerable complexity and code size, which increases the attack surface and resource usage in ways that are incompatible with space constraints. A leaner, purpose-built approach is therefore taken here, with wolfBoot's more advanced features noted as candidates for future work.

8.2. Security in space systems

8.2.1. GRBOOT

GRBOOT [23] is a space-compliant Boot Software product developed by Gaisler, the manufacturer of the LEON processor family. It originates from the GR712RC Boot Software designed specifically for the ESA JUICE mission instruments, where it was successfully deployed in eight LEON3FT-based instrument boards. That heritage product has since been generalised into a reusable bootloader supporting multiple hardware platforms.

GRBOOT implements the ESA SAVOIR Flight Computer Initialisation Sequence (SAVOIR-GS-002) and is developed in accordance with ECSS-E-ST-40C and ECSS-Q-ST-80C at software criticality category B. It currently targets the GR740, GR712RC and UT700 processor families, with ports available for the GR-CPCI-GR740, GR712RC development boards, and the UT700-LEAP. Boot memory options include parallel PROM, flash and similar non-volatile memories; application images can be loaded from memory-mapped storage (PROM, flash, MRAM, etc.) or from SPI flash.

Notable features include multi-processor support (SMP and AMP configurations), hardware system self-tests covering the CPU, L1/L2 caches, ROM and external memories, and a Boot Report that records self-test results and is made available to the loaded application. Application images use an ELF-like format with support for in-flight patching and optional compression, and integrity is checked before execution with failover on failure. The design cleanly separates Boot Memory from Application Storage Memory so that updating the application does not require updating the bootloader.

An optional SpaceWire/PUS Standby extension (available under a separate GRBOOT-STANDBY licence) adds a remote terminal over SpaceWire, PUS services in accordance with ECSS-E-ST-70-41C, remote uploading and patching of applications, and memory and register diagnostics. The product ships with fully automated test suites including unit tests that execute both on target hardware and in the TSIM3 LEON simulator, with code coverage captured in simulation.

8.2.2. N7 Space Boot Software

The N7 Space Boot Software [38] targets ARM Cortex-M and RISC-V microcontrollers — specifically the SAMV71Q21(RT), SAMRH71F20 and SAMRH707F18 — that are commonly used in smaller spacecraft and instrument electronics. Like GRBOOT, it is developed in compliance with ECSS-E-ST-40C and ECSS-Q-ST-80C and implements the SAVOIR Flight Computer Initialisation Sequence, and it has been produced during ESA contracts. It is pre-qualified for ECSS software criticality category B.

The communication interface uses a model-based PUS-C TC/TM stack with the data model defined

in ASN.1 and generated by the ESA asn1scc compiler. TC/TM links are available over UART and SpaceWire, with the possibility of extending to CAN and Ethernet. PUS-C services 1, 3, 5, 6, 9, 17 and 20 are supported. Self-test features cover bootloader image integrity, SRAM and SDRAM memory tests, and selected communication interfaces, with results delivered through a boot report and event telemetry. In-flight configurable application software image selection is supported through a Configuration Vector that controls image header locations, cache enable/disable, and self-test bypass. A demonstration version, with limited image size and disabled permanent reconfiguration, is available for evaluation on supported development boards.

Positioning relative to this work. Both GRBOOT and the N7 Space BSW are proprietary products and no internal source analysis was possible. From publicly available information, neither implements digital signature verification as part of the boot sequence. Application image integrity is checked, but the check is CRC- or hash-based and provides no authenticity guarantee against a deliberate adversary who controls the update channel. This is precisely the gap that SAVOIR-GS-002S and the PoC described in this thesis aim to address. The SAVOIR-GS-002S specification is formulated as an additive extension to SAVOIR-GS-002, meaning that a product such as GRBOOT or the N7 Space BSW could adopt the SEC requirements without replacing any of its existing functional requirements.

Conclusion

9.1. Summary of contributions

This thesis set out to investigate how Secure Boot can be incorporated into space mission bootloaders without compromising the strict reliability and performance constraints of the domain. The work progressed through five interconnected contributions.

A **lifecycle-based threat model** was developed, grounded in the ENISA Space Threat Landscape taxonomy [12]. Two attacker scenarios were identified as most relevant to the bootloader: a physical adversary with pre-launch hardware access and a remote adversary exploiting the communications and update channels after deployment. These scenarios motivated every design decision that followed.

A **formal Secure Boot specification** (SAVOIR-GS-002S extension) was produced, using the language and requirement structure of SAVOIR so that the document can be adopted or reviewed by the relevant standards bodies without requiring a translation step. The specification introduces seven security requirements (SEC.01–SEC.07) covering a hardware Root of Trust, digital signature verification of all ASW images, non-bypassability of the verification step, protection against classical and post-quantum adversaries, rollback protection via a monotonic counter, and structured failure reporting through existing telemetry channels. All seven requirements are additive to SAVOIR-GS-002 and can be adopted without replacing any existing requirements, with the sole exception of a constraint on BEF.22 (Fast Boot Path) that prevents signature verification from being placed in the skippable subset.

A **hardware trade-off analysis** was conducted to identify the minimal hardware primitives required to realise the specification. Two are strictly necessary: sector-level hardware-enforced write protection for the BSW and public key storage, and protected non-volatile storage for the monotonic rollback counter. Both are present in commodity ARM Cortex-M microcontrollers, such as the STM32F439ZI selected for the proof of concept, which implements them via Flash option bytes. A hardware hash accelerator is beneficial for classical schemes but not required.

A **proof-of-concept implementation** in C on the STM32F439ZI demonstrated that all seven SEC requirements are achievable on representative COTS hardware. The implementation includes A/B image partitioning, monotonic rollback protection in option-byte-protected non-volatile storage, PUS-compatible boot reporting, and an algorithm-agnostic cryptographic verification interface. An end-to-end automated test suite, communicating with the board over UART and SWD, verified the correct behaviour of all nominal and failure paths.

Finally, **performance benchmarks** for four signature schemes (ECDSA-P256, RSA-3072/4096, ML-DSA-44/65 and LMS) on the target hardware provided concrete data for mission designers choosing between classical and post-quantum algorithms. The results show that the security overhead introduced by signature verification is modest: even a hybrid ECDSA-P256 plus LMS deployment on a maximum-size 128 kB image completes in approximately 363 ms with hardware SHA-2 acceleration, well within a one-second boot budget.

9.2. Answers to research questions

The central question of this thesis was:

How can Secure Boot be incorporated into space mission bootloaders in a way that enhances security without compromising the strict reliability and performance constraints of space systems?

The answer demonstrated by this work is that Secure Boot can be incorporated by extending the existing SAVOIR-GS-002 framework with a layer of cryptographic verification requirements that are additive to, rather than in conflict with, the existing boot sequence structure. The key insight is that SAVOIR already mandates integrity checks, A/B partitioning and a structured nominal sequence; the security gap is that those checks are CRC-based and provide no authenticity guarantee against a deliberate adversary. Replacing CRC with digital signature verification, anchoring the verification key in hardware-write-protected memory, and adding a monotonic rollback counter in protected non-volatile storage are all that is needed to close this gap. The PoC on the STM32F439ZI confirms that these additions are feasible on widely available COTS hardware and that the resulting boot time overhead is acceptable even in the worst-case hybrid configuration.

How can Secure Boot be adapted using a secure-by-design approach compatible with space mission requirements? The specification follows a secure-by-design principle by making security failures non-recoverable within the boot sequence: if signature verification fails, the Boot SW rejects the image and reports the failure; there is no fallback path that permits unsigned code to run. The specification is compatible with space mission requirements because it extends SAVOIR-GS-002 without replacing it — existing mission software and ground toolchains continue to operate unchanged, with the only new ground-side obligation being the signing of firmware images before upload. The single exception is a constraint on the Fast Boot Path (BEF.22S) that prevents signature verification from being placed in the skippable subset, which is a minimal and justified restriction.

What is the minimal level of hardware support necessary for implementing Secure Boot in space-grade systems? Two hardware primitives are strictly necessary. First, sector-level write protection that is enforced by the Flash controller hardware and cannot be bypassed by software running at any privilege level, used to make the BSW code and the public key immutable at runtime. Second, protected non-volatile storage for the monotonic rollback counter, which must survive resets and must not be writable by the ASW. Both primitives are present in commodity ARM Cortex-M microcontrollers and do not require a dedicated secure enclave, TrustZone, or space-grade radiation-hardened silicon. A hardware hash accelerator improves throughput for classical schemes but is optional; the benchmarks show that all four evaluated schemes remain within a practical boot-time budget in software-only mode.

Which existing terrestrial Secure Boot standards and frameworks can be applied to space environments? NIST SP 800-193 [42] provides the most directly applicable normative framework. Its requirements for a hardware Root of Trust, an authenticated update mechanism, non-bypassability, and detection event logging map cleanly onto the SEC requirements defined in this work and are cited as the basis for each. The SUIT firmware update architecture [36] informed the image header format, with the header acting as the SUIT manifest and carrying the version identifier, image size, CRC and digital signature. Concepts from Android Verified Boot and wolfBoot — A/B partition selection, version-bound rollback protection, and algorithm-agnostic verification APIs — transferred well at the design level. Full adoption of UEFI Secure Boot or wolfBoot is not appropriate because their infrastructure complexity and code size are incompatible with the resource and audit requirements of space-grade software.

To what extent should post-quantum cryptographic algorithms be considered and do they necessitate dedicated hardware support? Post-quantum algorithms should be considered a mandatory element of the specification for any mission with a planned operational lifetime beyond approximately 2035, the NIST deadline for completing the post-quantum migration. The benchmark results demonstrate that both ML-DSA and LMS are viable on current COTS hardware without dedicated post-quantum hardware acceleration. ML-DSA-44 achieves 22 ms verification in software on a 1 kB image and remains competitive with classical schemes even at 128 kB. LMS requires only 3 kB of working

memory and 60 bytes for the public key, making it the most attractive option under extreme memory constraints and the only PQ scheme that BSI and ANSSI currently recommend for standalone use without hybridisation. No dedicated hardware support is needed for either scheme; the existing hardware hash accelerator provides no benefit for ML-DSA (which uses SHAKE-256 internally) and only modest gains for LMS. A hybrid deployment combining a classical scheme with a post-quantum scheme adds latency that is the sum of the two individual verification times; for ECDSA-P256 plus LMS on a 128 kB image with hardware SHA-2, this sum is approximately 363 ms, which is well within a one-second boot budget.

9.3. Future work

The directions described below were identified during the course of this work but were not pursued, primarily due to time constraints within the scope of a single research project. Some also require hardware that was not available during this work (e.g. open-source space-grade LEON processors). They are presented here as concrete next steps for anyone building on this foundation.

9.3.1. Modifying other standards to fully support Secure Boot

The secure boot design presented in this thesis operates at the Boot Software level, but it does not exist in isolation: the ECSS PUS standard [9] defines a set of ground-commanded memory management operations that can directly interfere with the boot chain if left unauthenticated. Fully supporting secure boot therefore requires that constraints be placed on specific ST[06] telecommands.

Service Type Interface Requirements. The following ST[06] telecommands must be subject to authentication and authorisation controls before they can be considered compatible with a secure boot implementation:

- **TC[6,2] / TC[6,11] Load raw memory data area** — allows arbitrary data to be written to any absolute address, including the Application Software storage region. An unauthenticated load command can overwrite a valid ASW image with a malicious one.
- **TC[6,1] Load object memory data** — equivalent risk for structured-addressed memories; allows overwriting named memory objects without cryptographic verification.
- **TC[6,16] Disable memory write protection** — removes hardware write protection from a specified memory region. Issuing this command before TC[6,2] is the standard two-step sequence to overwrite a protected region; without authentication, an attacker can unlock and replace the Boot Software itself.
- **TC[6,19] / TC[6,21] Load by reference** — load variants that source data from an on-board reference rather than inline in the telecommand. The referencing mechanism must be validated to prevent redirection to attacker-controlled memory.

All of these commands would need to carry an authenticated digest (e.g. a MAC or digital signature computed over the command parameters and payload) and be protected against replay attacks, for example by including a monotonic sequence number or timestamp verified by the onboard software before execution. Alternatively, they could be limited to specific memory regions such that they cannot interfere with the secure boot process.

9.3.2. Trust anchor management and key rotation (RFC 6024, RFC 5280)

The PoC stores a single public key compiled into the BSW at manufacturing time. This is the minimal viable Root of Trust, but it creates a single point of failure: if the corresponding private key is ever compromised, there is no in-field mechanism to rotate the trust anchor without reprogramming the BSW itself — an operation that requires write-protection to be disabled and a new, signed BSW to be uploaded. For a mission spanning a decade or more, this might be an unacceptable operational risk.

A more robust approach, motivated by the Trust Anchor Management requirements in RFC 6024 [41] and recommended as a key-compromise recovery technique in NIST SP 800-193 [42] (§3.5.1), introduces a hierarchical key structure. A root key — held offline and used rarely — authorises a set of subordinate update keys, which are the keys actually used to sign individual firmware images. Update

keys can then be rotated after launch by uploading a new key certificate signed by the root, without touching the BSW or the root trust anchor. The revocation mechanism defined in RFC 5280 [3] (X.509 certificate revocation lists) provides the complementary ability to invalidate a compromised update key before its scheduled expiry. Applying this model to the PoC would require adding a certificate parsing step to the BSW verification sequence and extending the image header to carry the update key certificate alongside the firmware signature, consistent with the layered manifest concept already defined in the SUIT architecture [36].

9.3.3. Partial image updates

The PoC transmits and verifies a complete signed ASW image for every update, regardless of the size of the change being applied. For missions where the ground-to-spacecraft link bandwidth is limited — as is common in deep-space operations — this is wasteful: a minor bug fix requires uploading the full 128 kB image rather than only the changed portion.

A segmented update scheme addresses this. The ASW image is divided into self-contained segments, each with its own header carrying a load address and an independent digital signature. An update then consists only of the segments that have changed; the BSW loads each segment into its correct RAM location and verifies its signature independently, leaving unchanged segments untouched in flash. This maps directly onto the component-based image structure defined in the SUIT manifest model [36, 35], which already accommodates partial updates. primary design trade-off is segment size. Since each segment header is dominated by the 4 kB signature field (sized to accommodate the ML-DSA-65 hybrid case), segments that are too small incur a large per-segment storage overhead; segments that are too large defeat the bandwidth saving. The non-linear relationship between image size and verification latency shown in Section 7.3 adds a further constraint, as the optimal segment size will differ across signature schemes.

9.3.4. Secure state management

The COMM partition in the PoC is a shared, unprotected flash region used to pass operational status from the ASW back to the BSW across a system reset. Because the COMM partition must be writable by the ASW, the BSW treats it as untrusted and never takes security-critical decisions based solely on the COMM value without first verifying digital signatures. This is a deliberate defence-in-depth decision, but it means that a compromised ASW can write any COMM value it chooses — for example, repeatedly signalling a swap request to trigger an iterative rollback, or preventing the BSW from entering standby mode when ground intervention is needed.

Authenticating the COMM partition would close this residual attack path, but no clean on-device solution exists. Generating an authenticated COMM value on-device requires a private key stored on the device, which a compromised ASW would also be able to access, so any scheme that relies on the ASW producing authenticated values is insecure. Schemes based on the ground pre-signing a fixed set of COMM state values at manufacturing time avoid this problem, but introduce a replay vulnerability: unless each signed payload includes a unique freshness token, a compromised ASW can replay any previously valid signed value. Including a sequence number inside the signed payload resolves the replay problem, but requires the ground to know at manufacturing time the exact sequence of COMM state transitions the spacecraft will make during its mission, which is not knowable in advance. Alternatively, freshly ground-signed COMM commands could be sent over the uplink whenever a state change is required, removing the manufacturing-time constraint entirely but making COMM state changes dependent on a live ground contact. The appropriate solution will depend on the mission's link availability and operational tempo; this is left as an open design problem.

9.3.5. Hardware security module and TrustZone integration

RFC 9397 [40] is an informational document regarding Trusted Execution Environment Provisioning. A Trusted Execution Environment is an environment that enforces that any code within the environment cannot be tampered with, and any data used by such code cannot be read or tampered with by any code outside the environment.

The Root of Trust in the PoC is implemented using Flash option bytes, which provide hardware-enforced write protection for BSW code and the stored public key. Because the BSW and ASW never execute

simultaneously — the BSW verifies and then transfers control, after which it is dormant — this write protection is sufficient to guarantee the integrity of the trust anchor: the ASW cannot overwrite BSW code or the public key. The remaining gap is one of confidentiality rather than integrity: a compromised ASW executing at the same processor privilege level can read BSW memory regions, including the stored public key. Reading the public key does not directly benefit an attacker in the current design (it is a public value by definition), but if private key material were ever moved on-device — for example, to support on-device COMM partition authentication or a TEEP-style TA provisioning model — such material would be equally readable by a compromised ASW. A TEE addresses this by enforcing a Secure World / Normal World split in hardware: Secure World memory is architecturally inaccessible to Normal World code regardless of privilege level, protecting both the integrity and the confidentiality of sensitive data at runtime, not only at rest.

The PoC was implemented on the STM32F439ZI, which predates the ARM TrustZone-M extension (introduced in Cortex-M23 and Cortex-M33). More recent microcontrollers such as the STM32H5 series include TrustZone-M alongside hardware SHA-3, ECDSA and RSA acceleration, which would allow the BSW and its cryptographic keys to be placed in Secure World memory that is architecturally inaccessible to the ASW. Vorago's next-generation VA5 family (Cortex-M55) also supports TrustZone and targets space-grade applications [53].

At the cutting-edge space-grade end, Xiphera has presented a hardware IP core specifically designed to provide post-quantum Secure Boot support for space-grade processors, including the Frontgrade GR765 [30]. This approach offloads cryptographic operations to a dedicated, formally verified IP block, providing stronger isolation and potentially faster verification than a software implementation on the main CPU.

This work deliberately chose the most accessible hardware in the ARM Cortex-M ecosystem to establish that the minimal hardware primitives required for Secure Boot are already present in proven COTS silicon. A natural next step is to validate the design on a TrustZone-enabled microcontroller, assess whether the stronger isolation guarantees justify the additional integration complexity, and repeat the performance benchmarks from Chapter 7 on the newer silicon to quantify the benefit of hardware-accelerated post-quantum operations.

9.3.6. Validation on LEON/space-grade hardware

All PoC benchmarks and functional tests were conducted on the STM32F439ZI. While that platform is representative of the ARM Cortex-M ecosystem and has been used in space flight software, SAVOIR was originally written with LEON/SPARC processors as the primary target — specifically the GR712RC, GR740 and the upcoming GR765 — and the canonical BSW implementation for that architecture is GRBOOT [23]. The framework has since been adopted more broadly: N7 Space has implemented a SAVOIR-compliant BSW on both RISC-V and ARM Cortex-M targets [38], which is the lineage most closely related to the platform used in this work. Until the SEC requirements are validated on a LEON processor, however, the claim that the specification is compatible with the canonical SAVOIR hardware environment remains unverified.

9.3.7. Formal verification of boot chain

The automated test suite described in Section 7.2 provides broad behavioural coverage but cannot prove the absence of bugs: it demonstrates that tested scenarios produce the expected outcome, but subtle state corruption under adversarial timing, unexpected transitions in the option-byte application sequence, or edge cases in the rollback logic that are difficult to express as concrete test inputs may remain undetected.

Formal methods would address this by exhaustively verifying that security-critical properties hold across all reachable states. The BSW state machine is well-suited to model checking: it has a small number of persistent states, clearly defined transitions and a concise set of invariants that must never be violated — for example, that unsigned firmware is never transferred to execution, or that the rollback counter is never decremented. A model-checked specification of the state machine could verify these properties formally, providing assurance that no reachable execution path bypasses the security checks. This would also bring the PoC closer to the formal correctness evidence required by ECSS-E-ST-40C for software at higher criticality categories.

9.3.8. Runtime detection beyond boot-time verification

The current design provides detection of unauthorised firmware exclusively at boot time: the Boot SW verifies each ASW image before transferring execution. Once the ASW is running, no further integrity checks are performed. NIST SP 800-193 [42] defines a broader *Detection* capability (§4.3) that includes integrity verification “at runtime” in addition to the pre-execution check. For spacecraft, this could mean periodic re-verification of ASW images in flash storage or the use of watchdog-timer-based health checks to detect unexpected corruption due to radiation-induced bit flips or a silent attacker who has modified flash memory without triggering a reboot. Extending the specification to cover a Root of Trust for Detection (RTD) that operates continuously during mission would bring the design closer to full NIST SP 800-193 compliance and increase resilience against in-operation attacks.

9.4. Reflection

9.4.1. Research in the broader computer science field

This work sits at the intersection of three areas that are each undergoing significant change: embedded systems security, post-quantum cryptographic engineering, and the application of terrestrial security practices to space systems. Spacecraft have always communicated with the ground, but modern missions increasingly rely on COTS components, standard networking protocols, and complex software stacks that are far more exposed and documented than the purpose-built systems of early space exploration. This has consistently revealed the same pattern seen in other safety-critical engineering domains: communities that have historically prioritised reliability over security accumulate a security debt that becomes increasingly costly to address as deployments scale and the incentives for attackers grow. The space sector is now beginning to face the same challenge, with the ENISA Space Threat Landscape [12] and ESA’s institutional commitment to cybersecurity [19, 15] marking a turning point analogous to what the industrial control systems community went through after incidents such as Stuxnet and the Ukrainian power grid attacks in the 2010s.

Within the computer science field more broadly, the contribution of this work is to provide concrete, measured evidence that modern cryptographic security mechanisms are deployable on severely resource-constrained embedded platforms. A common assumption in the embedded security literature is that cryptographic verification imposes prohibitive overhead; the benchmark results here show that this assumption does not hold for current hardware and algorithm choices. The implication is that the gap between what is possible and what is deployed in space is primarily an engineering and standardisation problem rather than a fundamental resource constraint.

On the post-quantum side, this work also contributes to an active area of research. The field is working through the implications of NIST’s recent post-quantum standardisation — including the stateful hash-based schemes LMS and XMSS (SP 800-208) and the PQC competition outputs ML-DSA and SLH-DSA (FIPS 204/205) — and a recurring question is whether these schemes are practical on resource-constrained hardware. The results presented here provide evidence for the affirmative: both ML-DSA and LMS are viable without dedicated hardware acceleration, with LMS being particularly attractive for the most constrained deployments. This adds a space-focused measurement to a body of literature that has mostly covered IoT and smart-card platforms.

9.4.2. Implications, applicability, and real-world impact

The most direct stakeholder for this work is ESA and the space engineering community. The specification is intentionally written in the style and language of SAVOIR so that it can be reviewed by the relevant working groups and potentially adopted as a formal extension to SAVOIR-GS-002. If adopted, it would provide mission designers with a clear, normative baseline for Secure Boot without requiring them to navigate the full complexity of terrestrial frameworks such as UEFI or Android Verified Boot, which are incompatible with space constraints. For existing missions based on GRBOOT or N7 Space BSW, the specification also identifies the precise gap — the absence of authenticated signature verification — that would need to be addressed to achieve the security properties described here.

For satellite operators and ground segment teams, the practical implication is that the attack vectors identified in the ENISA threat landscape [12] — firmware corruption and malicious code injection, whether through physical access before launch or through the communications channel after deploy-

ment — can be closed using COTS hardware without requiring a redesign of the spacecraft’s software architecture. The necessary security primitives are already present in mainstream microcontrollers; the main cost is the engineering effort to restructure the boot sequence and establish a key management process on the ground.

There are also limitations that mission designers must be aware of. The security guarantees provided by the design are contingent on the ground-side private key remaining secure. A compromised private key would allow an adversary to sign and install arbitrary firmware, defeating the entire chain of trust. This shifts the security problem from the spacecraft to the ground infrastructure, which is a more familiar and well-studied domain but is not addressed within the scope of this work. Additionally, the design provides boot-time integrity verification only; once the ASW is running, no further cryptographic checks are performed. A sufficiently capable adversary with persistent access to the running system could modify ASW code in RAM without triggering a reboot and therefore without triggering re-verification. Extending the design to cover runtime detection, as noted in Subsection 9.3.8, would address this residual risk.

Finally, the choice to evaluate on an ARM Cortex-M4 microcontroller reflects the most widely available and cost-accessible class of space-compatible processors today. The design principles transfer to LEON/SPARC platforms — which are the primary targets of SAVOIR — but the specific performance numbers, option byte mechanisms, and memory constraints are platform-specific. Validation on a GR processor, as discussed in Subsection 9.3.6, is therefore an important next step before the specification can be considered verified for its primary intended deployment context.

9.5. Use of AI

Generative AI tools were used during this project in accordance with the TU Delft guidelines on the use of generative AI in MSc end projects. The tools used were Claude (Anthropic) as the primary assistant and ChatGPT (OpenAI) on an occasional basis.

During the research phase, both tools were used to test ideas and explore counterarguments to proposed design decisions. During writing, they were used to correct grammar and spelling, improve sentence clarity, and suggest restructuring of drafts. During implementation, they helped navigate unfamiliar parts of existing codebases (wolfSSL, STM32 HAL), generate boilerplate code, assist with debugging, and produce code documentation.

All AI-generated output was manually reviewed and verified before inclusion. All research ideas, design decisions, arguments, interpretations, and conclusions are the author’s own.

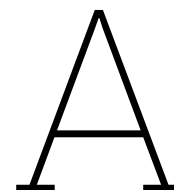
References

- [1] ANSSI. *ANSSI views on the Post-Quantum Cryptography transition (2023 follow up)*. 2023. url: https://messervices.cyber.gouv.fr/documents-guides/follow_up_position_paper_on_post_quantum_cryptography.pdf.
- [2] Jacob Beningo. *Bootloader Design for Microcontrollers in Embedded Systems*. 2015. url: https://www.beningo.com/wp-content/uploads/images/Papers/bootloader_design_for_microcontrollers_in_embedded_systems%20.pdf.
- [3] Sharon Boeyen et al. *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*. RFC 5280. May 2008. doi: 10.17487/RFC5280.
- [4] Scott O. Bradner. *Key words for use in RFCs to Indicate Requirement Levels*. RFC 2119. Mar. 1997. doi: 10.17487/RFC2119.
- [5] CCSDS. *CCSDS Cryptographic Algorithms*. 2019. url: https://ccsds.org/wp-content/uploads/gravity_forms/5-448e85c647331d9cbaf66c096458bdd5/2025/01//352x0b2.pdf?gv-iframe=true.
- [6] Angela Cratere et al. "On-Board Computer for CubeSats: State-of-the-Art and Future Trends". In: *IEEE Access* 12 (2024), pp. 99537–99569. doi: 10.1109/ACCESS.2024.3428388.
- [7] U.S.-CHINA ECONOMIC and SECURITY REVIEW COMMISSION. *2011 REPORT TO CONGRESS*. 2011. url: https://www.uscc.gov/sites/default/files/annual_reports/annual_report_full_11.pdf.
- [8] ECSS. *ECSS-E-ST-40C Rev.1 – Software (30 April 2025)*. 2025. url: <https://ecss.nl/standard/ecss-e-st-40c-rev-1-software-30-april-2025/> (visited on 01/12/2026).
- [9] ECSS. *ECSS-E-ST-70-41C – Telemetry and telecommand packet utilization (15 April 2016)*. 2016. url: <https://ecss.nl/standard/ecss-e-st-70-41c-space-engineering-telemetry-and-telecommand-packet-utilization-15-april-2016/> (visited on 01/30/2026).
- [10] ECSS. *ECSS-E-ST-80C – Space engineering – Security in space systems lifecycles (1 July 2024)*. 2024. url: <https://ecss.nl/standard/ecss-e-st-80c-space-engineering-security-in-space-systems-lifecycles/> (visited on 01/12/2026).
- [11] ECSS. *ECSS-Q-ST-80C Rev.2 – Software product assurance (30 April 2025)*. 2025. url: <https://ecss.nl/standard/ecss-q-st-80c-rev-2-software-product-assurance-30-april-2025/> (visited on 01/12/2026).
- [12] ENISA. *Space Threat Landscape*. Mar. 2025. url: https://www.enisa.europa.eu/sites/default/files/2025-03/Space_Threat_Landscape_Report_fin.pdf.
- [13] ESA. *ESA Space Shield*. 2023. url: <https://spaceshield.esa.int/> (visited on 11/13/2025).
- [14] ESA. *From custom-made to commercial: how ESA is changing the way that spacecraft are built*. 2020. url: https://www.esa.int/Enabling_Support/Preparing_for_the_Future/Discovery_and_Preparation/From_custom-made_to_commercial_how_ESA_is_changing_the_way_that_spacecraft_are_built (visited on 11/13/2025).
- [15] ESA Directorate of Technology, Engineering and Quality. *Cybersecurity – Cross-Cutting Initiative*. <https://technology.esa.int/program/cybersecurity>. Accessed: 2025.
- [16] Mohammed Eshaq et al. "CubeSat Flight Software: Insights and a Case Study". In: *Journal of Spacecraft and Rockets* 62.4 (2025), pp. 1328–1345. doi: 10.2514/1.A35882.
- [17] Mohammed Eshaq et al. "Flight Software Design and Implementation for a CubeSat". In: *2023 Advances in Science and Engineering Technology International Conferences (ASET)*. 2023, pp. 1–6. doi: 10.1109/ASET56582.2023.10180675.

- [18] European Space Agency. *ESA inaugurates new Cyber Security Operations Centre*. https://www.esa.int/Space_Safety/Cyber_resilience/ESA_inaugurates_new_Cyber_Security_Operations_Centre. May 2025.
- [19] European Space Agency. *ESA Technology Strategy, Version 1.0*. https://esamultimedia.esa.int/docs/technology/ESA_Technology_Strategy_Version_1_0.pdf. Accessed: 2025. 2023.
- [20] Gregory Falco. "The Vacuum of Space Cyber Security". In: *2018 AIAA SPACE and Astronautics Forum and Exposition* (2018). doi: 10.2514/6.2018-5275.
- [21] Gregory Falco et al. "An International Technical Standard for Commercial Space System Cyber-security - A Call to Action". In: *ASCEND 2022*. 2022. doi: 10.2514/6.2022-4302. url: <https://arc.aiaa.org/doi/abs/10.2514/6.2022-4302>.
- [22] Frontgrade Technologies. *Advanced RadHard Microprocessors*. 2026. url: <https://www.frontgrade.com/products/advanced-radhard-microprocessors> (visited on 05/21/2026).
- [23] Frontgrade Technologies. *GRBOOT*. 2025. url: <https://www.gaisler.com/products/grboot> (visited on 12/16/2025).
- [24] Frontgrade Technologies. *RadHard Microcontrollers*. 2026. url: <https://www.frontgrade.com/products/radiation-hardened-solutions-high-reliability-components/computing/microcontrollers> (visited on 05/21/2026).
- [25] Google. *Bootloader Overview*. Android Open Source Project. 2026. url: <https://source.android.com/docs/core/architecture/bootloader> (visited on 05/18/2026).
- [26] Google. *Implement OTA Updates*. Android Open Source Project. 2026. url: <https://source.android.com/docs/core/architecture/bootloader/updating> (visited on 05/18/2026).
- [27] Infineon Technologies. *Space Memory Portfolio: Radiation-Hardened Non-Volatile and Volatile Memory*. 2026. url: <https://www.infineon.com/products/high-reliability/space/rad-hard-memories> (visited on 05/20/2026).
- [28] Federal Office for Information Security. *Cryptographic Mechanisms: Recommendations and Key Lengths*. 2026. url: https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Publications/TechGuidelines/TG02102/BSI-TR-02102-1.pdf?__blob=publicationFile&v=10.
- [29] Federal Office for Information Security. *Status of quantum computer development*. 2025. url: https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/Publikationen/Studien/Quantenc omputer/Entwicklungsstand_QC_V_2_2.pdf?__blob=publicationFile&v=5.
- [30] Petri Jehkonen. "Post-Quantum Secure Boot for Space Infrastructures". In: *18th ESA Workshop on Avionics, Data, Control and Software Systems (ADCSS 2024)*. ESA/ESTEC, Noordwijk, The Netherlands, Oct. 2024. url: <https://indico.esa.int/event/532/contributions/9842/> (visited on 06/01/2026).
- [31] Hans Löhr, Ahmad-Reza Sadeghi, and Marcel Winandy. "Patterns for Secure Boot and Secure Storage in Computer Systems". In: *2010 International Conference on Availability, Reliability and Security*. IEEE, Feb. 2010, pp. 569–573. isbn: 978-1-4244-5879-0. doi: 10.1109/ARES.2010.110. url: <http://ieeexplore.ieee.org/document/5438035/> (visited on 11/28/2025).
- [32] Microchip Technology. *COTS-to-Radiation-Tolerant and Radiation-Hardened Devices*. 2026. url: <https://www.microchip.com/en-us/solutions/aerospace-and-defense/products/microcontrollers-and-microprocessors/cots-to-radiation-tolerant-and-radiation-hardened-devices> (visited on 05/20/2026).
- [33] Microsoft. *OEM*. <https://learn.microsoft.com/en-us/windows-hardware/design/device-experiences/oem-secure-boot>. Accessed: 2026.
- [34] Microsoft. *Windows Secure Boot*. <https://learn.microsoft.com/en-us/windows-hardware/manufacture/desktop/windows-secure-boot-key-creation-and-management-guidance?view=windows-11>. Accessed: 2026.
- [35] Brendan Moran, Hannes Tschofenig, and Henk Birkholz. *A Manifest Information Model for Firmware Updates in Internet of Things (IoT) Devices*. RFC 9124. Jan. 2022. doi: 10.17487/RFC9124.
- [36] Brendan Moran et al. *A Firmware Update Architecture for Internet of Things*. RFC 9019. Apr. 2021. doi: 10.17487/RFC9019.

- [37] Moutafis, Vissarion. *MORA: Hunting Space Bugs in your Sleep*. 2025. url: <https://repository.tudelft.nl/record/uuid:0a8ebae7-d6c0-4c19-acb1-22937316b033> (visited on 11/13/2025).
- [38] N7 Space. *N7 Space BSW*. 2025. url: <https://bootloader.space/> (visited on 12/16/2025).
- [39] NASA. *State-of-the-Art of Small Spacecraft Technology*. 2024. url: <https://www.nasa.gov/smallsat-institute/sst-soa/>.
- [40] Mingliang Pei et al. *Trusted Execution Environment Provisioning (TEEP) Architecture*. RFC 9397. July 2023. doi: 10.17487/RFC9397.
- [41] Raksha Reddy and Carl Wallace. *Trust Anchor Management Requirements*. RFC 6024. Oct. 2010. doi: 10.17487/RFC6024.
- [42] Andrew Regenscheid. *Platform Firmware Resiliency Guidelines*. Tech. rep. NIST SP 800-193. National Institute of Standards and Technology, May 2018. doi: 10.6028/NIST.SP.800-193. url: <https://doi.org/10.6028/NIST.SP.800-193>.
- [43] I. Aguilar Sanchez, G. Moury, and H. Weiss. “The CCSDS Space Data Link Security protocol”. In: *2010 - MILCOM 2010 MILITARY COMMUNICATIONS CONFERENCE*. 2010, pp. 219–224. doi: 10.1109/MILCOM.2010.5680491.
- [44] Koffi V. C. K. de Souza, Yassine Bouslimani, and Mohsen Ghribi. “Flight Software Development for a CubeSat Application”. In: *IEEE Journal on Miniaturization for Air and Space Systems* 3.4 (2022), pp. 184–196. doi: 10.1109/JMASS.2022.3206713.
- [45] National Institute of Standards and Technology. “Digital Signature Standard (DSS)”. In: *Federal Information Processing Standards Publication (FIPS) NIST FIPS 186-5* (2023). doi: 10.6028/NIST.FIPS.186-5.
- [46] National Institute of Standards and Technology. “Module-Lattice-Based Digital Signature Standard”. In: *Federal Information Processing Standards Publication (FIPS) NIST FIPS 204* (2024). doi: 10.6028/NIST.FIPS.204.
- [47] National Institute of Standards and Technology. “Recommendation for Stateful Hash-Based Signature Schemes”. In: *Special Publication NIST 800-208* (2020). doi: 10.6028/NIST.SP.800-208.
- [48] National Institute of Standards and Technology. “Stateless Hash-Based Digital Signature Standard”. In: *Federal Information Processing Standards Publication (FIPS) NIST FIPS 205* (2024). doi: 10.6028/NIST.FIPS.205.
- [49] National Institute of Standards and Technology. “The Keyed-Hash Message Authentication Code (HMAC)”. In: *Special Publication NIST 800-224* (2024). doi: 10.6028/NIST.SP.800-224.ipd.
- [50] 18 EU Member States. *Securing Tomorrow, Today: Transitioning to Post-Quantum Cryptography*. 2025. url: https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Crypto/PQC-joint-statement.pdf?__blob=publicationFile&v=5.
- [51] Terrailon, Jean-Loup. *SAVOIR*. 2021. url: <https://savoir.estec.esa.int/SAVOIRDocuments.htm> (visited on 02/13/2026).
- [52] UEFI. *UEFI Specification 2.10*. <https://uefi.org/specs/UEFI/2.10>. Accessed: 2026.
- [53] Vorago Technologies. *VORAGO Shatters Satellite Electronics Barriers, Launching Radiation-Tolerant Chips to Power the Next Generation of Satellite Constellations*. 2025. url: <https://www.voragotech.com/va4-powers-satellite-constellations> (visited on 05/21/2026).
- [54] Jianqiang Wang et al. “A comprehensive memory safety analysis of bootloaders”. In: *Proceedings 2025 Network and Distributed System Security Symposium* (2025). doi: 10.14722/ndss.2025.240330.
- [55] Johannes Willbold et al. “Space odyssey: An experimental software security analysis of satellites”. In: *2023 IEEE Symposium on Security and Privacy (SP)* (2023). doi: 10.1109/sp46215.2023.10351029.
- [56] WolfSSL. *LMS Versus XMSS Versus SLH-DSA*. 2025. url: <https://www.wolfssl.com/lms-versus-xmss-versus-slh-dsa/> (visited on 06/13/2026).
- [57] WolfSSL. *wolfBoot Secure Bootloader*. 2025. url: <https://www.wolfssl.com/products/wolfboot/> (visited on 01/19/2026).

- [58] Nikita Yadav et al. “Orbital Shield: Rethinking satellite security in the commercial off-the-shelf era”. In: *2024 Security for Space Systems (3S)* (2024), pp. 1–11. doi: 10.23919/3s60530.2024.10592292.



SAVOIR-GS-002S Secure Boot Generic Specification

All SAVOIR-GS-002 [51] requirements remain in force. Requirements carrying the suffix **S** (e.g. BEF.22S) modify a specific existing SAVOIR requirement; all remaining requirements are purely additive.

SAVOIR.BOOTSW.BEF.22S

Fast Boot Path — Digital Signature Verification is Non-Skippable

Notwithstanding SAVOIR.BOOTSW.BEF.22, the Boot SW shall not include digital signature verification of ASW images (as required by SAVOIR.BOOTSW.SEC.03) in the subset of Nominal Sequence steps that may be omitted during the Fast Boot Path sequence.

<i>Note:</i>	All other permitted omissions under BEF.22 remain unchanged.
<i>OptionInfo:</i>	OBC; PLM
<i>Rationale:</i>	Allowing signature verification to be skipped would create a well-defined bypass path for the Secure Boot chain, invalidating the authenticity guarantees of SEC.03–SEC.04.
<i>Verification Method:</i>	T

SAVOIR.BOOTSW.SEC.01

Root of Trust for Secure Boot

The Boot SW shall rely on a hardware-enforced Root of Trust to establish the authenticity and integrity of the Secure Boot process.

<i>Note:</i>	The Root of Trust is typically implemented using immutable memory or hardware security primitives.
<i>OptionInfo:</i>	OBC; PLM
<i>Assumption:</i>	Presence of boot memory with immutable regions
<i>Rationale:</i>	A trusted starting point is required to prevent compromise of the Secure Boot chain.
<i>Verification Method:</i>	ROD; I

SAVOIR.BOOTSW.SEC.02

Public Key Storage in Immutable Memory

The Boot SW shall use one or more (hashed) public keys stored in immutable memory to verify the digital signatures of ASW images.

<i>Note:</i>	Public keys stored in immutable memory cannot be altered by software updates. This key might be the root key of the Chain of Trust, with additional keys being used for software verification. A hash of a public key can be used to verify a key which is not immutable.
<i>OptionInfo:</i>	OBC; PLM
<i>Assumption:</i>	Boot memory provides immutable storage
<i>Rationale:</i>	Prevents unauthorized modification of trusted verification material.
<i>Verification Method:</i>	ROD; I

SAVOIR.BOOTSW.SEC.03

ASW Digital Signature Verification

The Boot SW shall perform a digital signature verification on all sections of the selected ASW image prior to execution.

<i>Note:</i>	Sections may include code, data, configuration, and metadata.
<i>OptionInfo:</i>	OBC; PLM
<i>Assumption:</i>	ASW image includes cryptographic signature
<i>Rationale:</i>	Ensures authenticity and integrity of the ASW before execution.
<i>Verification Method:</i>	T

SAVOIR.BOOTSW.SEC.04

Rejection of Unauthenticated ASW Images

The Boot SW shall reject and prevent execution of any ASW image that fails digital signature verification.

<i>Note:</i>	Failure includes invalid signature, unknown key, or revoked key usage.
<i>OptionInfo:</i>	OBC; PLM
<i>Assumption:</i>	Secure Boot verification is enabled
<i>Rationale:</i>	Prevents execution of tampered or unauthorized software.
<i>Verification Method:</i>	T

SAVOIR.BOOTSW.SEC.05

Protection against classical and post-quantum adversaries

The Boot SW shall support the use of a digital signature algorithm that provides resistance against both classical and quantum computational attacks.

<i>Note:</i>	Algorithm selection is mission-configurable. Hybridization is recommended, but a single algorithm may be used provided it offers resistance against both threat classes.
<i>OptionInfo:</i>	OBC; PLM
<i>Assumption:</i>	Availability of suitable cryptographic implementations
<i>Rationale:</i>	Mitigates long-term cryptographic risks for extended mission durations.
<i>Verification Method:</i>	ROD; T

SAVOIR.BOOTSW.SEC.06

Numeric Version Rollback Protection

The Boot SW shall permit execution only of ASW images whose version identifier lies within a pre-defined allowed rollback window. If the highest successfully executed version identifier is N and the permitted rollback window size is X , then only versions in the inclusive range:

$$[N, N - X]$$

shall be accepted for execution.

- For a rollback window of size $X = 1$, the Boot SW shall therefore accept only versions N and $N - 1$.
- Version identifiers are strictly positive integers.

<i>Note:</i>	ASW version identifiers are positive integers. Version tracking data shall be stored in protected non-volatile memory.
<i>OptionInfo:</i>	OBC; PLM
<i>Assumption:</i>	Protected resource retaining data across resets.
<i>Rationale:</i>	A configurable rollback window allows controlled recovery from recent software updates while preventing execution of outdated or potentially insecure versions. Limiting rollback to X consecutive versions mitigates unauthorized or unintended downgrades while permitting recovery within an operationally safe margin.
<i>Verification Method:</i>	T

SAVOIR.BOOTSW.SEC.07

Secure Boot Failure Reporting

The Boot SW shall report Secure Boot failures and their causes through telemetry or Boot Report mechanisms.

<i>Note:</i>	Failure causes include signature failure, key revocation, or version violation.
<i>OptionInfo:</i>	OBC; PLM
<i>Assumption:</i>	Boot Report or telemetry capability
<i>Rationale:</i>	Enables ground-based diagnosis and corrective actions.
<i>Verification Method:</i>	T; I

B

wolfSSL Benchmark Configuration

The benchmarks reported in Subsection 7.3.1 were produced using wolfSSL 5.8.2 on the STM32F439ZI (NUCLEO-F439ZI) with the configuration header below. The file is the STM32CubeIDE-generated `wolfSSL.I-CUBE-wolfSSL_conf.h`, modified to add the benchmark algorithm selection block at the bottom. All measurements were taken at a core clock of 168 MHz with ART accelerator and instruction cache enabled.

The key compile-time choices that directly affect the benchmark results are:

- **Math library:** `WOLF_CONF_MATH = 6` selects the portable `sp_int.c` multi-precision library with `WOLFSSL_SP_SMALL` to reduce flash usage at the cost of some speed.
- **ECC:** signing and key export are disabled (`NO_ECC_SIGN`, `NO_ECC_KEY_EXPORT`, `NO_ECC_DHE`) since only verification is needed.
- **RSA:** only the public-key / verify path is compiled (`WOLFSSL_RSA_PUBLIC_ONLY`, `WOLFSSL_RSA_VERIFY_INLINE`).
- **ML-DSA:** enabled via `HAVE_DILITHIUM` / `WOLFSSL_WC_DILITHIUM` with `WOLFSSL_DILITHIUM_VERIFY_ONLY` and `WOLFSSL_DILITHIUM_VERIFY_SMALL_MEM`; the level-87 parameter set is excluded (`WOLFSSL_NO_ML_DSA_87`). SHAKE-128/256 and SHA-3 are enabled only for ML-DSA builds.
- **LMS:** enabled via `WOLFSSL_HAVE_LMS` / `WOLFSSL_WC_LMS` with `WOLFSSL_LMS_VERIFY_ONLY`. The flag `WC_LMS_FULL_HASH` routes SHA-2 through the STM32 HASH peripheral; when commented out, software SHA-2 is used instead.
- **Hardware SHA-2:** the STM32F439xx target automatically undefines `NO_STM32_HASH`, enabling hardware-accelerated SHA-2 for ECDSA, RSA and LMS. ML-DSA uses SHAKE-256 which has no hardware support on this device.

Listing B.1: wolfSSL configuration header used for digital signature benchmarks

```
1 /* wolfSSL.I-CUBE-wolfSSL_conf.h - benchmark configuration
2  * wolfSSL 5.8.2 / STM32F439ZI @ 168 MHz
3  */
4
5 /*----- Core library mode -----*/
6 #define WOLF_CONF_WOLFCRYPT_ONLY 1 /* crypto-only, no TLS stack */
7 #define WOLF_CONF_MATH 6 /* sp_int.c (portable, small) */
8 #define WOLF_CONF_RTOS 1 /* bare-metal / single-threaded */
9 #define WOLF_CONF_RNG 0 /* RNG not needed for verify */
10
11 /*----- Classical algorithms -----*/
12 #define WOLF_CONF_RSA 1
13 #define WOLF_CONF_ECC 1
14 #define WOLF_CONF_DH 0
15
16 /*----- Hash algorithms -----*/
17 #define WOLF_CONF_SHA2_256 1 /* required by all schemes */
18 #define WOLF_CONF_SHA3 0 /* enabled per-build for ML-DSA */
```

```

19 #define WOLF_CONF_MD5 0
20 #define WOLF_CONF_SHA1 0
21 #define WOLF_CONF_SHA2_384 0
22 #define WOLF_CONF_SHA2_512 0
23
24 /*----- Unused features disabled -----*/
25 #define WOLF_CONF_AESGCM 0
26 #define WOLF_CONF_AESCBC 0
27 #define WOLF_CONF_CHAPOLY 0
28 #define WOLF_CONF_EDCURVE25519 0
29 #define WOLF_CONF_PSK 0
30 #define WOLF_CONF_PWDBASED 0
31 #define WOLF_CONF_KYBER 0
32 #define WOLF_CONF_ARMASM 0
33 #define WOLF_CONF_RESUMPTION 0
34
35 /* ----- Platform (STM32F439xx auto-enables HW SHA-2) ----- */
36 /* #define NO_STM32_HASH -- removed by platform block */
37
38 /* ----- RSA: verify-only path ----- */
39 #define WOLFSSL_RSA_PUBLIC_ONLY
40 #define WOLFSSL_RSA_VERIFY_INLINE
41 #define WC_RSA_PSS
42 #define WC_RSA_BLINDING
43
44 /* ----- ECC: verify-only path ----- */
45 #define NO_ECC_SIGN
46 #define NO_ECC_DHE
47 #define NO_ECC_KEY_EXPORT
48 #define ECC_USER_CURVES /* P-256 only */
49 #define ECC_SHAMIR
50 #define ECC_TIMING_RESISTANT
51 #define WOLFSSL_SP_MATH_ALL
52 #define SP_WORD_SIZE 32
53
54 /* =====
55  * Per-algorithm build flags (set BENCHMARK_ALGO at compile time)
56  * ===== */
57
58 /* Algorithm IDs (keep in sync with benchmark.h) */
59 #define ALGO_ECDSA 1
60 #define ALGO_RSA_3072 3
61 #define ALGO_RSA_4096 3 /* same flag, different key passed at runtime */
62 #define ALGO_ML_DSA_44 4
63 #define ALGO_ML_DSA_65 5
64 #define ALGO_LMS 6
65
66 /* ----- ML-DSA (BENCHMARK_ALGO == 4 or 5) ----- */
67 #if defined(BENCHMARK_ALGO) && \
68     (BENCHMARK_ALGO == ALGO_ML_DSA_44 || BENCHMARK_ALGO == ALGO_ML_DSA_65)
69     #define WOLFSSL_SHA3
70     #define WOLFSSL_SHAKE128
71     #define WOLFSSL_SHAKE256
72     #define HAVE_DILITHIUM
73     #define WOLFSSL_WC_DILITHIUM
74     #define WOLFSSL_DILITHIUM_ASSIGN_KEY
75     #define WOLFSSL_NO_ML_DSA_87 /* exclude level-87 to save flash */
76     #define WOLFSSL_DILITHIUM_VERIFY_ONLY
77     #define WOLFSSL_DILITHIUM_VERIFY_SMALL_MEM
78 #endif
79
80 /* ----- LMS (BENCHMARK_ALGO == 6) ----- */
81 #if defined(BENCHMARK_ALGO) && BENCHMARK_ALGO == ALGO_LMS
82     #define WOLFSSL_EXPERIMENTAL_SETTINGS
83     #define WOLFSSL_HAVE_LMS
84     #define WOLFSSL_LMS_VERIFY_ONLY
85     #define WOLFSSL_WC_LMS
86     /* Route SHA-2 through the STM32 HASH peripheral.
87      * Comment out to use software SHA-2 (SW SHA column). */
88     #define WC_LMS_FULL_HASH
89 #endif

```

